

算法岗

LeetCode 面试 100 题

从“会做题”到“会面试”：母题模型、推导、复杂度、边界与追问

2026 · Professional Interview Edition

定位 参考经典技术面试书的“问题驱动 -> 分析 -> 解法 -> 追问”讲解范式，全部正文、概括、代码与图示均为原创整理；不复刻《剑指 Offer》原文、例题叙述或版式。

适用岗位 算法工程师、机器学习工程师（MLE）、推荐/搜索/广告算法、NLP/CV、多模态、LLM/Agent 算法，以及需要 DSA Coding 轮的软件工程岗位。

100 题不是终点。真正要掌握的是约 25 个可迁移的算法原型。

出版说明与“真题”口径

本资料把“真题”分成两种证据等级：

- R (Reported)：公开面经中明确出现题、题号或高度同构题型。面经属于个人公开记录，不代表公司官方题库，也可能受 NDA 限制，因此只作为样本证据。
- H (High-frequency)：LeetCode 官方经典面试题、跨公司长期高频题型，或在多份公开经验中反复出现的母题。
- 所有题目叙述均为本资料的简化原创概括，用于训练“识别模型”和“现场推导”，不复制 LeetCode 或任何图书的受版权保护题面。

2026 观察 最新公开 MLE/算法岗经验仍显示 Coding 轮稳定存在：LinkedIn MLE 同时考 AI Coding 与 DSA；Meta SWE-ML 仍有多轮 Medium Coding；Google 公开面经继续出现 Top-K、拓扑、图与 DP。算法岗的正确策略不是弱化 DSA，而是把 DSA 与 ML/系统追问连接起来。

如何使用这本资料

1. 第一遍：只读每章的“识别信号”和“统一模板”，建立模式词典。
2. 第二遍：遮住解法，对每题用 3 分钟口述暴力解、关键不变量和最优复杂度。
3. 第三遍：核心 S 级题白板手写，目标 Medium 15-25 分钟、Hard 25-35 分钟。
4. 第四遍：只做“面试官追问”，把 LeetCode 题升级为数据流、内存受限、分布式或 ML 系统问题。
5. 复盘：记录失败原因而不是“是否 AC”：识别错、状态定义错、边界错、实现错、复杂度解释错。

六步白板协议

阶段	面试表达
1. 复述约束	输入规模、是否有序、是否允许修改、是否可能重复/负数/空输入。
2. 给出基线	先说一个正确但可能较慢的方案，建立可比较基准。
3. 找信息浪费	重复计算了什么？历史信息能否哈希？答案是否单调？局部状态能否复用？
4. 定义不变量	窗口何时合法？dfs 返回什么？heap 保存什么？dp[i] 精确表示什么？
5. 写代码并 dry-run	优先测空、单元素、重复值、极端边界、全负/全零、最大最小 K。
6. 主动给追问	复杂度、空间优化、流式/分布式、输入变化后算法是否仍成立。

25 个母题原型：100 题的真正骨架

母题原型	代表题	识别关键词
Hash lookup	1, 49	查补数 / 去重 / 映射
Prefix state	5, 6	连续区间 / 计数 / 历史状态
In-place mapping	7, 8	O(1) 空间 / 值域落在下标范围
Two pointers	12-14	有序 / 两端 / 三数和
Sliding window	15-20	连续子串 / 最长最短 / 至多 K
Fast/slow pointers	24-25	链表环 / 中点
Linked-list rewiring	21-30	反转 / 分组 / LRU
Monotonic stack/deque	34-35, 20	右侧第一个更大 / 窗口极值
Boundary binary	37-41	有序边界 / first / last
Answer binary	42-43	最小可行 / 最大可行 / check(x)
Heap Top-K	45-49	第 K / Top-K / k 路归并
Streaming state	48, 50	在线插入 / 随时查询
Sampling	51-52	权重随机 / 均匀映射
Tree DFS/BFS	54-65	层 / 子树 / 深度
Tree DP	57, 64	路径 / 直径 / 向父返回贡献
Grid components	66-67	岛屿 / 连通块
Topological sort	69-70	依赖 / 先修 / DAG
Union-Find	74-75	合并集合 / 动态连通
Graph shortest/reachability	71-77	最短步数 / 能否到达 / 权重关系
Backtracking	78-86	所有方案 / 排列组合 / 放置
Trie	85	前缀 / 多词搜索
Knapsack DP	89, 94	目标和 / 每个物品一次或无限次
Sequence DP	90-92	两个序列对齐 / LIS/LCS
State DP	88, 95-96	选/不选 / 最大最小双状态
Interval/Greedy	97-100	区间重叠 / 最少删除 / 最少跳跃

复杂度速查：面试中要能“口算”

结构	典型复杂度	面试备注
Hash/Set	平均 O(1) 查询/更新	最坏视实现而定；面试通常说明平均复杂度
排序	O(nlogn)	若只是 Top-K，不一定需要完整排序
Heap	push/pop O(logk)	Top-K 常保持 heap 大小为 K
BFS/DFS	O(V+E)	网格通常 O(mn)
二分	O(logR)	R 可是数组长度，也可以是答案范围
Union-Find	近 O(1)	严格为摊还 O(alpha(n))
DP	状态数 × 每状态转移数	先数状态，再数每个状态做多少工作
Backtracking	通常指数级	必须解释剪枝与输出规模，而非机械写

100 题目录

01 数组、哈希与前缀状态

1. LC1 两数之和; 2. LC49 字母异位词分组; 3. LC128 最长连续序列; 4. LC238 除自身以外数组的乘积; 5. LC560 和为 K 的子数组; 6. LC525 连续数组; 7. LC41 缺失的第一个正数; 8. LC73 矩阵置零; 9. LC54 螺旋矩阵; 10. LC380 $O(1)$ 插入删除与随机获取

02 双指针与滑动窗口

11. LC167 有序数组两数之和; 12. LC15 三数之和; 13. LC11 盛最多水的容器; 14. LC42 接雨水; 15. LC3 无重复字符的最长子串; 16. LC424 替换后的最长重复字符; 17. LC76 最小覆盖子串; 18. LC438 找到字符串中所有字母异位词; 19. LC209 长度最小的子数组; 20. LC239 滑动窗口最大值

03 链表与缓存设计

21. LC206 反转链表; 22. LC92 反转链表 II; 23. LC21 合并两个有序链表; 24. LC141 环形链表; 25. LC142 环形链表 II; 26. LC25 K 个一组翻转链表; 27. LC143 重排链表; 28. LC148 排序链表; 29. LC138 复制带随机指针的链表; 30. LC146 LRU 缓存

04 栈、解析器与单调结构

31. LC20 有效的括号; 32. LC155 最小栈; 33. LC394 字符串解码; 34. LC739 每日温度; 35. LC84 柱状图中最大的矩形; 36. LC224 基本计算器

05 二分搜索与答案空间

37. LC704 二分查找; 38. LC33 搜索旋转排序数组; 39. LC153 寻找旋转排序数组中的最小值; 40. LC34 查找元素首尾位置; 41. LC4 寻找两个正序数组的中位数; 42. LC875 爱吃香蕉的珂珂; 43. LC1011 在 D 天内送达包裹的能力; 44. LC162 寻找峰值

06 堆、Top-K、流式计算与随机采样

45. LC215 数组中的第 K 个最大元素; 46. LC347 前 K 个高频元素; 47. LC973 最接近原点的 K 个点; 48. LC295 数据流的中位数; 49. LC23 合并 K 个升序链表; 50. LC346 数据流中的移动平均值; 51. LC528 按权重随机选择; 52. LC470 用 Rand7 实现 Rand10; 53. LC912 排序数组

07 二叉树与 Tree DP

54. LC104 二叉树最大深度; 55. LC102 二叉树层序遍历; 56. LC226 翻转二叉树; 57. LC543 二叉树的直径; 58. LC110 平衡二叉树; 59. LC98 验证二叉搜索树; 60. LC230 BST 中第 K 小的元素; 61. LC236 二叉树最近公共祖先; 62. LC199 二叉树右视图; 63. LC105 前序与中序构造二叉树; 64. LC124 二叉树中的最大路径和; 65. LC314 二叉树垂序遍历

08 图、BFS/DFS、拓扑与并查集

66. LC200 岛屿数量; 67. LC695 岛屿的最大面积; 68. LC133 克隆图; 69. LC207 课程表; 70. LC210 课程表 II; 71. LC127 单词接龙; 72. LC994 腐烂的橘子; 73. LC417 太平洋大西洋水流问题; 74. LC721 账户合并; 75. LC684 冗余连接; 76. LC841 钥匙和房间; 77. LC399 除法求值

09 回溯、搜索树与 Trie

78. LC78 子集; 79. LC46 全排列; 80. LC39 组合总和; 81. LC90 子集 II; 82. LC79 单词搜索; 83. LC131 分割回文串; 84. LC51 N 皇后; 85. LC208 实现 Trie; 86. LC698 划分为 K 个相等子集

10 动态规划、贪心与区间

87. LC70 爬楼梯; 88. LC198 打家劫舍; 89. LC322 零钱兑换; 90. LC300 最长递增子序列; 91. LC1143 最长公共子序列; 92. LC72 编辑距离; 93. LC139 单词拆分; 94. LC416 分割等和子集; 95. LC152 乘积最大子数组; 96. LC121 买卖股票的最佳时机; 97. LC56 合并区间; 98. LC57 插入区间; 99. LC435 无重叠区间; 100. LC45 跳跃游戏 II

第 1 章 数组、哈希与前缀状态

把“枚举历史”变成“查询状态”。识别信号：查补数、计数、连续区间、原地索引映射。

统一识别 先写暴力的两层循环，再问：过去的信息能否用一个 O(1) 查询结构压缩？

核心原型：Hash / Prefix State / In-place Mapping

```
# 哈希/前缀状态统一模板
freq = {0: 1}
state = 0
for x in data:
    state = update(state, x)
    answer += query(freq, state)
    freq[state] = freq.get(state, 0) + 1
```

题号	LeetCode	难度	级别	核心模式
1	1	Easy	S	Hash 查补数
2	49	Medium	S	Canonical Key
3	128	Medium	S	Hash Set + 起点判定
4	238	Medium	S	Prefix/Suffix Product
5	560	Medium	S	Prefix Sum + Hash Count
6	525	Medium	A	Prefix State Compression
7	41	Hard	S	In-place Index Mapping
8	73	Medium	A	State Reuse
9	54	Medium	A	Boundary Simulation
10	380	Medium	A	Hash + Dynamic Array

题 001 · LC 1 两数之和 / Two Sum

Easy 优先级 S H 高频母题 Hash 查补数

题意概括 给定数组与目标值，返回一对下标，使两数之和等于目标。

核心突破 扫描到 x 时，只需要知道 target-x 是否已经出现；把“向后找另一个数”改成“向历史查补数”。

最优路径 维护 value->index；先查补数，再写入当前值，避免同一元素被使用两次。

复杂度 O(n) 时间 / O(n) 空间

高频失误 先写入再查询在 target=2*x 时可能误用自身。

追问升级 若数组有序？用双指针；若要所有不重复数对？排序去重；若数据流？只保留历史哈希。

参考实现 (Python)

```
def two_sum(nums, target):
    pos = {}
    for i, x in enumerate(nums):
        need = target - x
        if need in pos:
            return [pos[need], i]
        pos[x] = i
    return []
```

30 秒口述 “我先给一个基线解法，然后利用 Hash 查补数 去掉重复工作。关键不变量是：扫描到 x 时，只需要知道 $\text{target}-x$ 是否已经出现；把“向后找另一个数”改成“向历史查补数”。最终复杂度是 $O(n)$ 时间 / $O(n)$ 空间。”

题 002 · LC 49 字母异位词分组 / Group Anagrams

Medium 优先级 S H 高频母题 Canonical Key

题意概括 把由相同字符组成、顺序不同的字符串放在同一组。

核心突破 关键不是两两比较，而是设计“顺序无关但内容唯一”的规范化键。

最优路径 可排序字符得到 key；若字符集固定，用 26 维频次数组转 tuple，更接近 $O(\text{total chars})$ 。

复杂度 $O(\text{总字符数})$ 或排序版 $O(\sum L \log L)$

高频失误 用可变 list 当哈希键；忽略 Unicode/字符集假设。

追问升级 若字符串极长？频次键；若字符集巨大？稀疏计数；如何做分布式分组？

题 003 · LC 128 最长连续序列 / Longest Consecutive Sequence

Medium 优先级 S H 高频母题 Hash Set + 起点判定

题意概括 无序数组中找最长的连续整数序列长度。

核心突破 只从“没有前驱 $x-1$ ”的元素开始向右扩展，每个元素最多被真正遍历一次。

最优路径 把所有数放 set；对每个 x ，若 $x-1$ 不在 set，则不断查 $x+1$ 。

复杂度 $O(n)$ 平均时间 / $O(n)$ 空间

高频失误 从每个元素都向右扫会退化到 $O(n^2)$ 。

追问升级 若内存受限？排序；若要求在线更新？并查集/区间合并。

白板实现检查点 先写出 Hash Set + 起点判定的不变量，再把边界条件直接编码进循环条件；完成后用一个正常样例 + 一个极端样例 dry-run。

30 秒口述 “我先给一个基线解法，然后利用 Hash Set + 起点判定 去掉重复工作。关键不变量是：只从“没有前驱 $x-1$ ”的元素开始向右扩展，每个元素最多被真正遍历一次。最终复杂度是 $O(n)$ 平均时间 / $O(n)$ 空间。”

题 004 · LC 238 除自身以外数组的乘积 / Product of Array Except Self

Medium 优先级 S H 高频母题 Prefix/Suffix Product

题意概括 不使用除法，为每个位置求其他元素的乘积。

核心突破 答案等于左侧乘积乘右侧乘积；右侧可以用一个滚动变量，避免额外 suffix 数组。

最优路径 第一遍写左前缀到 ans；第二遍从右向左维护 suffix 并乘入。

复杂度 $O(n)$ 时间 / $O(1)$ 额外空间（不计输出）

高频失误 零元素、负数；把输出数组错误计入额外空间。

追问升级 若允许除法？讨论 0 的特殊处理；若数值溢出？模运算/大整数。

题 005 · LC 560 和为 K 的子数组 / Subarray Sum Equals K

Medium 优先级 S H 高频母题 Prefix Sum + Hash Count

题意概括 统计和为 K 的连续子数组数量。

核心突破 若当前前缀和为 s，需要历史前缀 s-k；哈希表保存“出现次数”而非只有是否出现。

最优路径 freq[0]=1；扫描累加 s；答案加 freq[s-k]；再 freq[s]++。

复杂度 O(n) 时间 / O(n) 空间

高频失误 忘记 freq[0]=1；先更新 freq[s] 会影响 k=0。

追问升级 改成 XOR=K、可被 K 整除、0/1 数量相等分别如何定义前缀状态？

参考实现 (Python)

```
from collections import defaultdict

def subarray_sum(nums, k):
    freq = defaultdict(int)
    freq[0] = 1
    prefix = ans = 0
    for x in nums:
        prefix += x
        ans += freq[prefix - k]
        freq[prefix] += 1
    return ans
```

30 秒口述 “我先给一个基线解法，然后利用 Prefix Sum + Hash Count 去掉重复工作。关键不变量是：若当前前缀和为 s，需要历史前缀 s-k；哈希表保存“出现次数”而非只有是否出现。最终复杂度是 O(n) 时间 / O(n) 空间。”

题 006 · LC 525 连续数组 / Contiguous Array

Medium 优先级 A H 高频母题 Prefix State Compression

题意概括 0/1 数组中找 0 和 1 数量相同的最长连续区间。

核心突破 把 0 视作 -1；相同前缀和两次出现，其间区间和为 0。为最大长度要保存最早位置。

最优路径 first={0:-1}；prefix += 1 或 -1；若见过则更新最大距离，否则记录首次下标。

复杂度 O(n) / O(n)

高频失误 保存最近位置会损失最长答案。

追问升级 若 0/1/2 三类数量相同？用二维差分状态。

题 007 · LC 41 缺失的第一个正数 / First Missing Positive

Hard 优先级 S H 高频母题 In-place Index Mapping

题意概括 在线性时间、常数额外空间下找数组中缺失的最小正整数。

核心突破 答案必在 [1,n+1]；把值 x 放到下标 x-1，让数组本身充当哈希表。

最优路径 循环交换直到 nums[i] 不在 [1,n] 或已经在正确位置；最后找首个 nums[i]!=i+1。

复杂度 O(n) / O(1)

高频失误 交换条件不防重复值会死循环。

追问升级 若不能修改输入？位图/哈希；为什么答案上界一定是 n+1？

白板实现检查点 先写出 In-place Index Mapping 的不变量，再把边界条件直接编码进循环条件；完成后用一个正常样例 + 一个极端样例 dry-run。

30 秒口述 “我先给一个基线解法，然后利用 In-place Index Mapping 去掉重复工作。关键不变量是：答案必在 $[1, n+1]$ ；把值 x 放到下标 $x-1$ ，让数组本身充当哈希表。最终复杂度是 $O(n) / O(1)$ 。”

题 008 · LC 73 矩阵置零 / Set Matrix Zeroes

Medium 优先级 A H 高频母题 State Reuse

题意概括 若某元素为 0，将其整行整列置 0，要求常数额外空间。

核心突破 用第一行和第一列作为行列标记，但二者共享 (0,0)，因此额外记录第一列是否应清零。

最优路径 先记录 first_col；扫描内部写标记；再按标记置零；最后处理第一行/列。

复杂度 $O(mn) / O(1)$

高频失误 更新顺序导致标记被提前覆盖。

追问升级 如何做到单遍？为什么通常不值得；稀疏矩阵如何处理？

题 009 · LC 54 螺旋矩阵 / Spiral Matrix

Medium 优先级 A H 高频母题 Boundary Simulation

题意概括 按顺时针螺旋顺序输出矩阵。

核心突破 维护 top, bottom, left, right 四个不变量，每走完一条边就收缩边界。

最优路径 依次上边、右边、下边、左边；进入后两条边前检查边界仍有效。

复杂度 $O(mn) / O(1)$

高频失误 单行或单列时重复访问。

追问升级 若要原地旋转矩阵？若输入是流式行数据？

题 010 · LC 380 O(1) 插入删除与随机获取 / Insert Delete GetRandom O(1)

Medium 优先级 A H 高频母题 Hash + Dynamic Array

题意概括 设计集合，插入、删除、随机取值平均 $O(1)$ 。

核心突破 数组支持 $O(1)$ 随机访问；哈希记录下标；删除时把末尾元素搬到洞位。

最优路径 insert append；remove swap-with-last + pop；getRandom 随机数组下标。

复杂度 平均 $O(1)$ 每操作 / $O(n)$

高频失误 删除末尾元素时索引更新顺序；重复插入。

追问升级 允许重复元素如何设计 RandomizedCollection？并发如何保证一致性？

第 2 章 双指针与滑动窗口

处理连续区间与有序结构的核心武器。识别信号：最长/最短子串、至多 K 次、区间合法性。

统一识别 定义窗口合法性 invariant；right 扩张，left 只在必要时单调右移。

核心原型：Two Pointers / Sliding Window

```
# 变长滑动窗口
left = 0
for right, x in enumerate(data):
    add(x)
    while not valid():
        remove(data[left])
        left += 1
    update_answer(left, right)
```

题号	LeetCode	难度	级别	核心模式
11	167	Medium	A	Opposite Pointers
12	15	Medium	S	Sort + Two Pointers
13	11	Medium	S	Greedy Two Pointers
14	42	Hard	S	Two Pointers / Monotonic Stack
15	3	Medium	S	Variable Window
16	424	Medium	A	Window Invariant
17	76	Hard	S	Need/Have Sliding Window
18	438	Medium	A	Fixed Window
19	209	Medium	A	Positive-number Window
20	239	Hard	S	Monotonic Deque

题 011 · LC 167 有序数组两数之和 / Two Sum II

Medium 优先级 A H 高频母题 Opposite Pointers

题意概括 有序数组中找两数之和等于目标。

核心突破 若和太小，只能增大左值；和太大，只能减小右值，单调性保证不漏解。

最优路径 $l=0, r=n-1$ ；比较 $nums[l]+nums[r]$ 并移动一端。

复杂度 $O(n) / O(1)$

高频失误 下标是否要求 1-based；移动错误一端。

追问升级 为什么 Hash 不是最优空间？扩展到 3Sum。

题 012 · LC 15 三数之和 / 3Sum

Medium 优先级 S H 高频母题 Sort + Two Pointers

题意概括 找出所有和为 0 的不重复三元组。

核心突破 排序固定第一个数后，剩余问题变成 Two Sum II；去重必须分别处理 i、left、right。

最优路径 排序；枚举 i；双指针收缩；命中后跳过重复值。

复杂度 $O(n^2) / O(1)$ 除输出

高频失误 漏去重或过度去重； $i > 0$ 且 $nums[i] == nums[i-1]$ 才跳。

追问升级 4Sum 如何泛化为 k-Sum？如何做剪枝？

参考实现 (Python)

```
def three_sum(nums):
    nums.sort(); ans = []
    for i in range(len(nums)-2):
        if i and nums[i] == nums[i-1]: continue
        if nums[i] > 0: break
        l, r = i+1, len(nums)-1
        while l < r:
            s = nums[i] + nums[l] + nums[r]
            if s < 0: l += 1
            elif s > 0: r -= 1
            else:
                ans.append([nums[i], nums[l], nums[r]])
                l += 1; r -= 1
                while l < r and nums[l] == nums[l-1]: l += 1
                while l < r and nums[r] == nums[r+1]: r -= 1
    return ans
```

30 秒口述 “我先给一个基线解法，然后利用 Sort + Two Pointers 去掉重复工作。关键不变量是：排序固定第一个数后，剩余问题变成 Two Sum II；去重必须分别处理 i、left、right。最终复杂度是 $O(n^2) / O(1)$ 除输出。”

题 013 · LC 11 盛最多水的容器 / Container With Most Water

Medium 优先级 S H 高频母题 Greedy Two Pointers

题意概括 两条竖线形成容器，求最大面积。

核心突破 面积受短板限制；若移动长板，宽度下降而短板不变，不可能改善，因此必须移动短板。

最优路径 从两端开始，更新面积，移动高度较小的一端。

复杂度 $O(n) / O(1)$

高频失误 无法解释“为什么移动短板”只背代码。

追问升级 若允许删除 K 条线？是否仍可贪心？

白板实现检查点 先写出 Greedy Two Pointers 的不变量，再把边界条件直接编码进循环条件；完成后用一个正常样例 + 一个极端样例 dry-run。

30 秒口述 “我先给一个基线解法，然后利用 Greedy Two Pointers 去掉重复工作。关键不变量是：面积受短板限制；若移动长板，宽度下降而短板不变，不可能改善，因此必须移动短板。最终复杂度是 $O(n) / O(1)$ 。”

题 014 · LC 42 接雨水 / Trapping Rain Water

Hard 优先级 S R 公开面经样本 Two Pointers / Monotonic Stack

题意概括 柱状图中统计可接雨水总量。

核心突破 双指针版本维护 left_max/right_max；较小一侧的水位已经由其最大值确定。

最优路径 比较 left_max 与 right_max，推进较小一侧并累积 max-height。

复杂度 $O(n) / O(1)$

高频失误 直接比较 `height[l], height[r]` 时要保持正确不变量。

追问升级 用单调栈如何解释“横向结算”？二维接雨水需要什么结构？

白板实现检查点 先写出 Two Pointers / Monotonic Stack 的不变量，再把边界条件直接编码进循环条件；完成后用一个正常样例 + 一个极端样例 dry-run。

30 秒口述 “我先给一个基线解法，然后利用 Two Pointers / Monotonic Stack 去掉重复工作。关键不变量是：双指针版本维护 `left_max/right_max`；较小一侧的水位已经由其最大值确定。最终复杂度是 $O(n) / O(1)$ 。”

题 015 · LC 3 无重复字符的最长子串 / Longest Substring Without Repeating Characters

Medium 优先级 **S** H 高频母题 Variable Window

题意概括 找不含重复字符的最长连续子串。

核心突破 窗口合法性：每个字符最多出现一次；可用 last index 让 left 一步跳过冲突位置。

最优路径 维护 `last[ch]`；`left = max(left, last[ch] + 1)`；更新 last 和最大长度。

复杂度 $O(n) / O(\text{字符集})$

高频失误 left 不能向左回退，必须 max。

追问升级 至多 K 种字符？恰好 K 种？

参考实现 (Python)

```
def length_of_longest_substring(s):
    last = {}; left = ans = 0
    for right, ch in enumerate(s):
        if ch in last:
            left = max(left, last[ch] + 1)
        last[ch] = right
        ans = max(ans, right - left + 1)
    return ans
```

30 秒口述 “我先给一个基线解法，然后利用 Variable Window 去掉重复工作。关键不变量是：窗口合法性：每个字符最多出现一次；可用 last index 让 left 一步跳过冲突位置。最终复杂度是 $O(n) / O(\text{字符集})$ 。”

题 016 · LC 424 替换后的最长重复字符 / Longest Repeating Character Replacement

Medium 优先级 **A** H 高频母题 Window Invariant

题意概括 最多替换 K 个字符，使某段全部相同，求最大长度。

核心突破 窗口需要替换的最少字符数 = `window_len - max_freq`；只要 $\leq K$ 就合法。

最优路径 扩 right 更新计数与历史 `max_freq`；非法时移动 left。

复杂度 $O(n) / O(\text{字符集})$

高频失误 纠结 `max_freq` 是否需要随 left 减小；求最大长度时可保持历史最大。

追问升级 若要返回具体区间？若字符替换代价不同？

题 017 · LC 76 最小覆盖子串 / Minimum Window Substring

Hard 优先级 **S** H 高频母题 Need/Have Sliding Window

题意概括 找 s 中覆盖 t 所有字符及次数的最短子串。

核心突破 窗口有两套计数：need 与 window；用 valid 记录满足需求的字符种类数，避免每次全表比较。

最优路径 right 扩张；满足全部需求后 while 收缩 left 并更新最优。

复杂度 $O(\text{len}(s) + \text{len}(t)) / O(\text{字符集})$

高频失误 valid 的定义是“种类满足”不是总字符数；更新顺序易错。

追问升级 若要所有最短窗口？若 t 是词集合而非字符？

参考实现 (Python)

```
from collections import Counter, defaultdict

def min_window(s, t):
    need = Counter(t); window = defaultdict(int)
    required = len(need); valid = 0
    left = 0; best = (float('inf'), 0, 0)
    for right, ch in enumerate(s):
        window[ch] += 1
        if ch in need and window[ch] == need[ch]: valid += 1
        while valid == required:
            if right-left+1 < best[0]: best = (right-left+1, left, right+1)
            c = s[left]; left += 1
            if c in need and window[c] == need[c]: valid -= 1
            window[c] -= 1
        return ' ' if best[0] == float('inf') else s[best[1]:best[2]]
```

30 秒口述 “我先给一个基线解法，然后利用 Need/Have Sliding Window 去掉重复工作。关键不变量是：窗口有两套计数：need 与 window；用 valid 记录满足需求的字符种类数，避免每次全表比较。最终复杂度是 $O(\text{len}(s) + \text{len}(t)) / O(\text{字符集})$ 。”

题 018 · LC 438 找到字符串中所有字母异位词 / Find All Anagrams in a String

Medium 优先级 A H 高频母题 Fixed Window

题意概括 找 s 中所有与 p 字母频次相同的长度固定子串起点。

核心突破 窗口长度固定为 len(p)，因此每次 right 前进后把超长的 left 移出即可。

最优路径 维护计数或差分数组；长度到达 m 时判断并滑动。

复杂度 $O(n) / O(\text{字符集})$

高频失误 每个窗口重新排序/计数导致 $O(nm)$ 。

追问升级 如何用一个 mismatch 计数做到 $O(1)$ 判断？

题 019 · LC 209 长度最小的子数组 / Minimum Size Subarray Sum

Medium 优先级 A H 高频母题 Positive-number Window

题意概括 正整数数组中找和至少 target 的最短连续子数组。

核心突破 正数保证 left 右移只会让和变小，形成单调窗口；若有负数则失效。

最优路径 right 累加；while sum >= target 更新答案并减去 nums[left]。

复杂度 $O(n) / O(1)$

高频失误 忽略“全为正数”这一关键条件。

追问升级 含负数时可用前缀和 + 单调队列（LC 862）。

题 020 · LC 239 滑动窗口最大值 / Sliding Window Maximum

Hard 优先级 **S** H 高频母题 Monotonic Deque

题意概括 长度 K 的每个窗口求最大值。

核心突破 维护下标递减的双端队列；队首永远是当前最大值，队尾去掉不可能再成为最大值的元素。

最优路径 移除过期队首；弹出 $\leq$ 当前值的队尾；加入当前下标；窗口形成后读队首。

复杂度 $O(n) / O(k)$

高频失误 队列存值会难以判断过期；相等元素处理不当。

追问升级 实时流式窗口如何做？如果查询最大和最小同时？

参考实现 (Python)

```
from collections import deque

def max_sliding_window(nums, k):
    q = deque(); ans = []
    for i, x in enumerate(nums):
        while q and q[0] <= i-k: q.popleft()
        while q and nums[q[-1]] <= x: q.pop()
        q.append(i)
        if i >= k-1: ans.append(nums[q[0]])
    return ans
```

30 秒口述 “我先给一个基线解法，然后利用 Monotonic Deque 去掉重复工作。关键不变量是：维护下标递减的双端队列；队首永远是当前最大值，队尾去掉不可能再成为最大值的元素。最终复杂度是 $O(n) / O(k)$ 。”

第 3 章 链表与缓存设计

考察指针不变量、局部重连和数据结构组合。算法岗常从链表继续追问缓存、在线系统与并发。

统一识别 画出节点关系再写代码；任何一次 next 改写前先保存后继。

核心原型：Pointer Rewiring / Hash + DLL

```
# 链表安全重连
prev, cur = None, head
while cur:
    nxt = cur.next      # 先保后继
    cur.next = prev     # 再改指针
    prev, cur = cur, nxt
```

题号	LeetCode	难度	级别	核心模式
21	206	Easy	S	Pointer Rewiring
22	92	Medium	S	Segment Reversal
23	21	Easy	S	Two-list Merge
24	141	Easy	S	Floyd Cycle Detection
25	142	Medium	S	Floyd Entrance Math
26	25	Hard	S	Chunked Reversal
27	143	Medium	A	Middle + Reverse + Merge
28	148	Medium	A	Merge Sort on List
29	138	Medium	A	Node Mapping / Interleaving
30	146	Medium	S	Hash + Doubly Linked List

题 021 · LC 206 反转链表 / Reverse Linked List

Easy 优先级 **S** R 公开面经样本 Pointer Rewiring

题意概括 反转单链表并返回新头节点。

核心突破 每一步只需要 prev、cur、next 三个指针；先保存 next，再改 cur.next。

最优路径 循环：nxt=cur.next; cur.next=prev; prev=cur; cur=nxt。

复杂度 O(n) / O(1)

高频失误 改 next 前没保存后继导致链表丢失。

追问升级 递归版空间复杂度？如何反转区间？

面经证据 公开 TikTok MLE phone screen 记录：先反转链表，再追问区间反转（LC 92 同题）。

参考实现 (Python)

```
def reverse_list(head):
    prev, cur = None, head
    while cur:
        nxt = cur.next
        cur.next = prev
        prev, cur = cur, nxt
```

```
prev, cur = cur, nxt
return prev
```

30 秒口述 “我先给一个基线解法，然后利用 Pointer Rewiring 去掉重复工作。关键不变量是：每一步只需要 prev、cur、next 三个指针；先保存 next，再改 cur.next。最终复杂度是 $O(n) / O(1)$ 。”

题 022 · LC 92 反转链表 II / Reverse Linked List II

Medium 优先级 S R 公开面经样本 Segment Reversal

题意概括 只反转位置 [left,right] 的节点。

核心突破 用 dummy 统一处理 left=1；定位反转区间前驱，然后做头插法或局部反转。

最优路径 pre 走到 left-1；重复把 curr.next 摘下并插到 pre 后。

复杂度 $O(n) / O(1)$

高频失误 区间边界 off-by-one；left=1。

追问升级 继续追问 K 个一组反转。

面经证据 公开 TikTok MLE phone screen 记录直接给出 Reverse Linked List II。

白板实现检查点 先写出 Segment Reversal 的不变量，再把边界条件直接编码进循环条件；完成后用一个正常样例 + 一个极端样例 dry-run。

30 秒口述 “我先给一个基线解法，然后利用 Segment Reversal 去掉重复工作。关键不变量是：用 dummy 统一处理 left=1；定位反转区间前驱，然后做头插法或局部反转。最终复杂度是 $O(n) / O(1)$ 。”

题 023 · LC 21 合并两个有序链表 / Merge Two Sorted Lists

Easy 优先级 S H 高频母题 Two-list Merge

题意概括 合并两个升序链表。

核心突破 dummy + tail 让头节点特殊情况消失；每次接较小节点。

最优路径 比较 l1.val/l2.val；tail.next 指向小者并推进；最后接剩余。

复杂度 $O(m+n) / O(1)$

高频失误 忘记接剩余链表。

追问升级 K 个链表：堆 vs 分治。

题 024 · LC 141 环形链表 / Linked List Cycle

Easy 优先级 S H 高频母题 Floyd Cycle Detection

题意概括 判断链表是否存在环。

核心突破 快慢指针若有环必相遇；若无环快指针到 null。

最优路径 slow 每次 1 步，fast 每次 2 步。

复杂度 $O(n) / O(1)$

高频失误 循环条件 fast and fast.next。

追问升级 为什么一定相遇？如何求环入口？

白板实现检查点 先写出 Floyd Cycle Detection 的不变量，再把边界条件直接编码进循环条件；完成后用一个正常样例 + 一个极端样例 dry-run。

30 秒口述 “我先给一个基线解法，然后利用 Floyd Cycle Detection 去掉重复工作。关键不变量是：快慢指针若有环必相遇；若无环快指针到 null。最终复杂度是 $O(n) / O(1)$ 。”

题 025 · LC 142 环形链表 II / Linked List Cycle II

Medium 优先级 S H 高频母题 Floyd Entrance Math

题意概括 若有环，返回环的入口节点。

核心突破 第一次相遇后，一个指针回到头部，两者每次一步，再次相遇即入口。

最优路径 先 Floyd 求 meeting；若无环返回； $p=head, q=meeting$ 同速前进。

复杂度 $O(n) / O(1)$

高频失误 只记结论不会推导 $a+c = \text{整数倍环长}$ 。

追问升级 求环长？两个链表是否相交？

白板实现检查点 先写出 Floyd Entrance Math 的不变量，再把边界条件直接编码进循环条件；完成后用一个正常样例 + 一个极端样例 dry-run。

30 秒口述 “我先给一个基线解法，然后利用 Floyd Entrance Math 去掉重复工作。关键不变量是：第一次相遇后，一个指针回到头部，两者每次一步，再次相遇即入口。最终复杂度是 $O(n) / O(1)$ 。”

题 026 · LC 25 K 个一组翻转链表 / Reverse Nodes in k-Group

Hard 优先级 S H 高频母题 Chunked Reversal

题意概括 每 K 个节点一组反转，不足 K 个保持原样。

核心突破 每轮先确认存在 K 个节点，再反转 $[groupPrev.next, kth]$ ，最后更新组间连接。

最优路径 找 kth；保存 groupNext；局部反转直到 groupNext；groupPrev 移到旧组头。

复杂度 $O(n) / O(1)$

高频失误 组边界与 $kth.next$ 保存顺序。

追问升级 若最后不足 K 也要反转？若 K 动态变化？

白板实现检查点 先写出 Chunked Reversal 的不变量，再把边界条件直接编码进循环条件；完成后用一个正常样例 + 一个极端样例 dry-run。

30 秒口述 “我先给一个基线解法，然后利用 Chunked Reversal 去掉重复工作。关键不变量是：每轮先确认存在 K 个节点，再反转 $[groupPrev.next, kth]$ ，最后更新组间连接。最终复杂度是 $O(n) / O(1)$ 。”

题 027 · LC 143 重排链表 / Reorder List

Medium 优先级 A H 高频母题 Middle + Reverse + Merge

题意概括 将链表重排为 $L_0, L_n, L_1, L_{n-1}, \dots$

核心突破 分解成三道母题：找中点、反转后半段、交替合并。

最优路径 Floyd 找中点；断开；reverse second；交错 merge。

复杂度 $O(n) / O(1)$

高频失误 奇偶长度断点；忘记断开造成环。

追问升级 如何不修改节点值；能否递归？

题 028 · LC 148 排序链表 / Sort List

Medium 优先级 A H 高频母题 Merge Sort on List

题意概括 $O(n \log n)$ 时间、尽量低额外空间排序链表。

核心突破 链表随机访问差但合并 $O(1)$ 重连，因此归并排序天然适配。

最优路径 快慢指针拆半；递归排序左右；merge。

复杂度 $O(n \log n)$ 时间 / $O(\log n)$ 递归栈

高频失误 拆分时 slow/fast 起点导致无法真正分半。

追问升级 自底向上归并可做到 $O(1)$ 辅助空间。

题 029 · LC 138 复制带随机指针的链表 / Copy List with Random Pointer

Medium 优先级 A H 高频母题 Node Mapping / Interleaving

题意概括 深拷贝每个节点拥有 next 与 random 指针的链表。

核心突破 Hash 版建立 old->new； $O(1)$ 额外空间版把新节点插到旧节点后面，让 random 可通过 old.random.next 找到。

最优路径 三遍：穿插复制；连 random；拆分两条链表。

复杂度 $O(n)$ / $O(1)$ 额外空间

高频失误 拆分时破坏原链表；random=None。

追问升级 图的深拷贝为何是同一个思想？

题 030 · LC 146 LRU 缓存 / LRU Cache

Medium 优先级 S R 公开面经样本 Hash + Doubly Linked List

题意概括 设计 get/put 均 $O(1)$ 的最近最少使用缓存。

核心突破 Hash 提供 $O(1)$ 定位；双向链表提供 $O(1)$ 删除与移动；头部最近使用，尾部最久未用。

最优路径 get 命中后移到头；put 更新/插入；超容量删除尾节点并删 hash。

复杂度 平均 $O(1)$ / $O(\text{capacity})$

高频失误 双向链表边界、更新已有 key 时容量处理。

追问升级 LFU、TTL、多线程、分片、GPU KV Cache eviction 如何扩展？

参考实现 (Python)

```
# 核心结构: dict + 双向链表。
# 伪代码强调接口与不变量:
get(k):
    if k not in map: return -1
    move_to_front(map[k])
    return map[k].value

put(k,v):
    if k exists: update + move_to_front
    else: insert_front(new_node); map[k]=node
    if size > capacity: evict tail.prev and delete map entry
```

30 秒口述 “我先给一个基线解法，然后利用 Hash + Doubly Linked List 去掉重复工作。关键不变量是：Hash 提供 $O(1)$ 定位；双向链表提供 $O(1)$ 删除与移动；头部最近使用，尾部最久未用。最终复杂度是 平均 $O(1)$ / $O(\text{capacity})$ 。”

第 4 章 栈、解析器与单调结构

栈不只是括号匹配。它用于表达“尚未解决的历史状态”、递归语法和最近更大/更小元素。

统一识别 问自己：当前元素到来后，能一次性解决哪些历史元素？

核心原型：Stack / Monotonic Stack / Parser

```
# 单调栈：当前元素结算历史
stack = []
for i, x in enumerate(a):
    while stack and violates(a[stack[-1]], x):
        j = stack.pop()
        settle(j, i)
    stack.append(i)
```

题号	LeetCode	难度	级别	核心模式
31	20	Easy	S	Stack Matching
32	155	Medium	S	Augmented State
33	394	Medium	S	Stack Parser
34	739	Medium	S	Monotonic Stack
35	84	Hard	S	Monotonic Increasing Stack
36	224	Hard	A	Expression Parsing

题 031 · LC 20 有效的括号 / Valid Parentheses

Easy 优先级 S H 高频母题 Stack Matching

题意概括 判断多种括号是否正确嵌套。

核心突破 右括号必须匹配“最近一个尚未闭合”的左括号，天然 LIFO。

最优路径 遇左括号 push；遇右括号检查栈顶；结束要求栈空。

复杂度 $O(n)$ / $O(n)$

高频失误 空栈 pop；只比较数量不比较嵌套。

追问升级 带通配符 * 如何处理？

白板实现检查点 先写出 Stack Matching 的不变量，再把边界条件直接编码进循环条件；完成后用一个正常样例 + 一个极端样例 dry-run。

30 秒口述 “我先给一个基线解法，然后利用 Stack Matching 去掉重复工作。关键不变量是：右括号必须匹配“最近一个尚未闭合”的左括号，天然 LIFO。最终复杂度是 $O(n)$ / $O(n)$ 。”

题 032 · LC 155 最小栈 / Min Stack

Medium 优先级 S H 高频母题 Augmented State

题意概括 设计支持 push/pop/top/getMin 均 $O(1)$ 的栈。

核心突破 每个节点同时记录“到这里为止的最小值”，用空间换查询时间。

最优路径 push(x,min(x,current_min)); pop 同步弹。

复杂度 $O(1)$ 每操作 / $O(n)$

高频失误 重复最小值时只保存一个全局 min 会出错。

追问升级 如何用一个栈编码差值减少空间？

题 033 · LC 394 字符串解码 / Decode String

Medium 优先级 **S** H 高频母题 Stack Parser

题意概括 解析 k[encoded] 的嵌套编码。

核心突破 遇 [时保存外层字符串与重复次数；遇] 时恢复最近未完成上下文。

最优路径 num 累数字； '[' push(curr,num) 并清空； ']' pop 并拼接。

复杂度 $O(\text{输出长度}) / O(\text{嵌套深度} + \text{输出})$

高频失误 多位数字、嵌套；复杂度不能只写 $O(n)$ 因输出可能膨胀。

追问升级 如何写递归下降解析器？

题 034 · LC 739 每日温度 / Daily Temperatures

Medium 优先级 **S** H 高频母题 Monotonic Stack

题意概括 对每一天找右侧第一个更高温度距离。

核心突破 栈保存“还没找到更高温”的下标，并保持温度单调递减。

最优路径 对 i: while $T[i] > T[\text{stack}[-1]]$, 弹出 j 并 $\text{ans}[j] = i - j$; 最后 push i。

复杂度 $O(n) / O(n)$

高频失误 栈存温度而非下标会丢距离。

追问升级 Next Greater Element、股票跨度、直方图如何统一？

参考实现 (Python)

```
def daily_temperatures(t):
    ans = [0] * len(t); st = []
    for i, x in enumerate(t):
        while st and x > t[st[-1]]:
            j = st.pop(); ans[j] = i - j
        st.append(i)
    return ans
```

30 秒口述 “我先给一个基线解法，然后利用 Monotonic Stack 去掉重复工作。关键不变量是：栈保存“还没找到更高温”的下标，并保持温度单调递减。最终复杂度是 $O(n) / O(n)$ 。”

题 035 · LC 84 柱状图中最大的矩形 / Largest Rectangle in Histogram

Hard 优先级 **S** H 高频母题 Monotonic Increasing Stack

题意概括 柱状图中求最大矩形面积。

核心突破 某柱被更矮柱弹出时，右边界确定；弹栈后新的栈顶给出左侧第一个更矮位置。

最优路径 末尾加哨兵 0；维持递增下标栈；弹出 h 后宽度 $= i - \text{stack}[-1] - 1$ 。

复杂度 $O(n) / O(n)$

高频失误 宽度公式；没有哨兵导致尾部未结算。

追问升级 Maximal Rectangle 如何逐行转成 histogram?

白板实现检查点 先写出 Monotonic Increasing Stack 的不变量，再把边界条件直接编码进循环条件；完成后用一个正常样例 + 一个极端样例 dry-run。

30 秒口述 “我先给一个基线解法，然后利用 Monotonic Increasing Stack 去掉重复工作。关键不变量是：某柱被更矮柱弹出时，右边界确定；弹栈后新的栈顶给出左侧第一个更矮位置。最终复杂度是 $O(n)$ / $O(n)$ 。”

题 036 · LC 224 基本计算器 / Basic Calculator

Hard 优先级 A H 高频母题 Expression Parsing

题意概括 计算含 +,-,括号与空格的表达式。

核心突破 括号表示新的符号上下文；可以栈保存进入括号前的 result 和 sign。

最优路径 扫描数字累加；+/- 提交前一个数；'(' push(result,sign); ')' 合并。

复杂度 $O(n)$ / $O(\text{括号深度})$

高频失误 一元负号、连续空格、多位数。

追问升级 加入 */ 后如何做 precedence? 用 shunting-yard 或递归下降。

第 5 章 二分搜索与答案空间

高级二分的重点不是 sorted array，而是“可行性随答案单调变化”。

统一识别 先定义 check(x)，再证明 False...False True...True 或相反的单调结构。

核心原型：Boundary Binary Search / Binary Search on Answer

```
# 搜索最小可行答案
lo, hi = lower_bound, upper_bound
while lo < hi:
    mid = lo + (hi - lo) // 2
    if feasible(mid):
        hi = mid
    else:
        lo = mid + 1
return lo
```

题号	LeetCode	难度	级别	核心模式
37	704	Easy	S	Exact Binary Search
38	33	Medium	S	Partitioned Monotonicity
39	153	Medium	A	Boundary Binary Search
40	34	Medium	S	Lower/Upper Bound
41	4	Hard	S	Binary Partition
42	875	Medium	S	Binary Search on Answer
43	1011	Medium	S	Feasibility Search
44	162	Medium	A	Slope Binary Search

题 037 · LC 704 二分查找 / Binary Search

Easy 优先级 S H 高频母题 Exact Binary Search

题意概括 在升序数组中查找目标下标。

核心突破 真正要练的是区间定义：闭区间 $[l, r]$ 或半开 $[l, r)$ ，必须从循环条件到更新保持一致。

最优路径 闭区间：while $l \leq r$ ；mid；小则 $l = \text{mid} + 1$ ，大则 $r = \text{mid} - 1$ 。

复杂度 $O(\log n)$ / $O(1)$

高频失误 混用闭区间与半开区间。

追问升级 写 lower_bound / upper_bound。

白板实现检查点 先写出 Exact Binary Search 的不变量，再把边界条件直接编码进循环条件；完成后用一个正常样例 + 一个极端样例 dry-run。

30 秒口述 “我先给一个基线解法，然后利用 Exact Binary Search 去掉重复工作。关键不变量是：真正要练的是区间定义：闭区间 $[l, r]$ 或半开 $[l, r)$ ，必须从循环条件到更新保持一致。最终复杂度是 $O(\log n)$ / $O(1)$ 。”

题 038 · LC 33 搜索旋转排序数组 / Search in Rotated Sorted Array

Medium 优先级 S H 高频母题 Partitioned Monotonicity

题意概括 升序数组旋转后查目标，要求 $O(\log n)$ 。

核心突破 每次 mid 至少有一侧仍然有序；判断 target 是否落在有序侧决定保留哪半。

最优路径 若 $\text{nums}[l] \leq \text{nums}[mid]$ 左侧有序，否则右侧有序；再做区间包含判断。

复杂度 $O(\log n) / O(1)$

高频失误 重复元素会破坏简单判断；边界包含关系。

追问升级 有重复元素对应 LC81，最坏为何退化 $O(n)$ ？

参考实现 (Python)

```
def search_rotated(nums, target):
    l, r = 0, len(nums)-1
    while l <= r:
        m = (l+r)//2
        if nums[m] == target: return m
        if nums[l] <= nums[m]:
            if nums[l] <= target < nums[m]: r = m-1
            else: l = m+1
        else:
            if nums[m] < target <= nums[r]: l = m+1
            else: r = m-1
    return -1
```

30 秒口述 “我先给一个基线解法，然后利用 Partitioned Monotonicity 去掉重复工作。关键不变量是：每次 mid 至少有一侧仍然有序；判断 target 是否落在有序侧决定保留哪半。最终复杂度是 $O(\log n) / O(1)$ 。”

题 039 · LC 153 寻找旋转排序数组中的最小值 / Find Minimum in Rotated Sorted Array

Medium 优先级 A H 高频母题 Boundary Binary Search

题意概括 无重复旋转数组中找最小值。

核心突破 比较 mid 与 right: $\text{mid} > \text{right}$ 说明最小值在右侧，否则最小值在 mid 或左侧。

最优路径 while $l < r$; mid ; 若 $\text{nums}[\text{mid}] > \text{nums}[\text{r}]$ $l = \text{mid} + 1$ else $r = \text{mid}$ 。

复杂度 $O(\log n) / O(1)$

高频失误 $r = \text{mid} - 1$ 会跳过可能的最小值 mid。

追问升级 有重复值时如何处理？

题 040 · LC 34 查找元素首尾位置 / Find First and Last Position

Medium 优先级 S H 高频母题 Lower/Upper Bound

题意概括 在有序数组中找目标的起止位置。

核心突破 把“找一个 target”升级为找两个边界：第一个 $\geq \text{target}$ 与第一个 $> \text{target}$ 。

最优路径 写统一 lower_bound(x); $\text{left} = \text{lb}(\text{target})$, $\text{right} = \text{lb}(\text{target} + \text{epsilon}) - 1$ 或 upper_bound-1。

复杂度 $O(\log n) / O(1)$

高频失误 目标不存在时的验证；整数溢出时不要 $\text{target} + 1$ 。

追问升级 泛化为 C++ lower_bound/upper_bound 语义。

白板实现检查点 先写出 Lower/Upper Bound 的不变量，再把边界条件直接编码进循环条件；完成后用一个正常样例 + 一个极端样例 dry-run。

30 秒口述 “我先给一个基线解法，然后利用 Lower/Upper Bound 去掉重复工作。关键不变量是：把“找一个 target”升级为找两个边界：第一个 $\geq \text{target}$ 与第一个 $> \text{target}$ 。最终复杂度是 $O(\log n) / O(1)$ 。”

题 041 · LC 4 寻找两个正序数组的中位数 / Median of Two Sorted Arrays

Hard 优先级 S R 公开面经样本 Binary Partition

题意概括 两个有序数组总长度的中位数，要求对较短数组 $O(\log m)$ 。

核心突破 在短数组切一刀，长数组切的位置由总左半长度决定；只需满足 $\text{leftA} \leq \text{rightB}$ 且 $\text{leftB} \leq \text{rightA}$ 。

最优路径 二分 partitionA； $\text{partitionB} = \text{half-partitionA}$ ；根据边界关系移动；合法后取 $\max(\text{left}, \min(\text{right}))$ 。

复杂度 $O(\log \min(m, n)) / O(1)$

高频失误 哨兵 $\pm \text{inf}$ 、奇偶总长度、二分短数组。

追问升级 扩展到第 K 小；为什么不是简单 merge？

面经证据 公开 Alibaba 面经曾要求两个有序数组中位数并强调对数复杂度。

白板实现检查点 先写出 Binary Partition 的不变量，再把边界条件直接编码进循环条件；完成后用一个正常样例 + 一个极端样例 dry-run。

30 秒口述 “我先给一个基线解法，然后利用 Binary Partition 去掉重复工作。关键不变量是：在短数组切一刀，长数组切的位置由总左半长度决定；只需满足 $\text{leftA} \leq \text{rightB}$ 且 $\text{leftB} \leq \text{rightA}$ 。最终复杂度是 $O(\log \min(m, n)) / O(1)$ 。”

题 042 · LC 875 爱吃香蕉的珂珂 / Koko Eating Bananas

Medium 优先级 S H 高频母题 Binary Search on Answer

题意概括 给定若干堆和总小时 H，求最小整数速度。

核心突破 速度越大所需时间越少， $\text{check}(\text{speed}) \leq H$ 具有单调性。

最优路径 搜索 $[1, \max(\text{pile})]$ ； $\text{hours} = \sum(\text{ceil}(p/\text{speed}))$ ；可行则收缩右界。

复杂度 $O(n \log M) / O(1)$

高频失误 ceil 用 $(p+k-1)/k$ ；边界返回 l。

追问升级 机器处理速率、吞吐、延迟 SLA 类题都可映射。

参考实现 (Python)

```
def min_eating_speed(piles, h):
    lo, hi = 1, max(piles)
    while lo < hi:
        mid = (lo+hi)//2
        hours = sum((p+mid-1)//mid for p in piles)
        if hours <= h: hi = mid
        else: lo = mid+1
    return lo
```

30 秒口述 “我先给一个基线解法，然后利用 Binary Search on Answer 去掉重复工作。关键不变量是：速度越大所需时间越少， $\text{check}(\text{speed}) \leq H$ 具有单调性。最终复杂度是 $O(n \log M) / O(1)$ 。”

题 043 · LC 1011 在 D 天内送达包裹的能力 / Capacity To Ship Packages Within D Days

Medium 优先级 S H 高频母题 Feasibility Search

题意概括 保持顺序装船，在 D 天内送完的最小船容量。

核心突破 容量下界 $\max(\text{weights})$ ，上界 $\sum(\text{weights})$ ；容量越大需要天数不增。

最优路径 $\text{check}(\text{cap})$ 模拟按顺序装箱并统计天数；二分最小可行 cap 。

复杂度 $O(n \log \sum W) / O(1)$

高频失误 下界写 0 会让单件超容量； check 中天数初始化。

追问升级 拆成 K 段最小化最大段和（LC410）。

题 044 · LC 162 寻找峰值 / Find Peak Element

Medium 优先级 A H 高频母题 Slope Binary Search

题意概括 相邻元素不同，找任意峰值。

核心突破 比较 mid 与 $\text{mid}+1$ 相当于看局部斜率；上坡说明右侧必有峰，下坡说明左侧含 mid 必有峰。

最优路径 $\text{while } l < r; \text{mid}; \text{nums}[\text{mid}] < \text{nums}[\text{mid}+1]$ 则 $l = \text{mid}+1$ 否则 $r = \text{mid}$ 。

复杂度 $O(\log n) / O(1)$

高频失误 不要访问 $\text{mid}-1/\text{mid}+1$ 两侧导致边界复杂化。

追问升级 二维峰值如何做？

第 6 章 堆、Top-K、流式计算与随机采样

与推荐、检索、召回、在线统计最贴近。把离线排序升级为在线、内存受限、分布式版本。

统一识别 先问是否真的需要全排序；Top-K 通常只需要 K 大小的堆或 QuickSelect。

核心原型：Heap / QuickSelect / Streaming / Sampling

```
# 流式 Top-K (最小堆)
heap = []
for x in stream:
    heappush(heap, x)
    if len(heap) > k:
        heappop(heap)
# heap 保存当前全局 K 大候选
```

题号	LeetCode	难度	级别	核心模式
45	215	Medium	S	Heap / QuickSelect
46	347	Medium	S	Frequency + Selection
47	973	Medium	S	Top-K / QuickSelect
48	295	Hard	S	Two Heaps
49	23	Hard	S	K-way Merge
50	346	Easy	A	Fixed-size Stream State
51	528	Medium	S	Prefix Weight + Sampling
52	470	Medium	S	Rejection Sampling
53	912	Medium	S	Sorting Implementation

题 045 · LC 215 数组中的第 K 个最大元素 / Kth Largest Element in an Array

Medium 优先级 **S** H 高频母题 Heap / QuickSelect

题意概括 找数组第 K 大，不要求排序输出。

核心突破 不必全排序：维护 K 大小最小堆 $O(n\log k)$ ，或 QuickSelect 平均 $O(n)$ 。

最优路径 堆版：push, size>K 时 pop；QuickSelect 通过 partition 只递归一侧。

复杂度 $O(n\log k)$ / $O(k)$, QuickSelect 平均 $O(n)$

高频失误 “第 K 大”与索引 $n-k$ ；QuickSelect 最坏 $O(n^2)$ 。

追问升级 十亿数据、外存、100 台机器如何做 Top-K？

参考实现 (Python)

```
import heapq

def find_kth_largest(nums, k):
    heap = []
    for x in nums:
        heapq.heappush(heap, x)
        if len(heap) > k:
            heapq.heappop(heap)
    return heap[0]
```

30 秒口述 “我先给一个基线解法，然后利用 Heap / QuickSelect 去掉重复工作。关键不变量是：不必全排序：维护 K 大小最小堆 $O(n\log k)$ ，或 QuickSelect 平均 $O(n)$ 。最终复杂度是 $O(n\log k) / O(k)$ ，QuickSelect 平均 $O(n)$ 。”

题 046 · LC 347 前 K 个高频元素 / Top K Frequent Elements

Medium 优先级 S H 高频母题 Frequency + Selection

题意概括 返回出现频率最高的 K 个不同元素。

核心突破 先计数，再在“元素-频率”上做堆、桶排序或 QuickSelect；频率上界是 n，桶解法可 $O(n)$ 。

最优路径 Counter；bucket[freq].append(x)；从高频桶逆序收集到 K。

复杂度 $O(n) / O(n)$

高频失误 直接对原数组排序浪费；K 是不同元素数。

追问升级 大规模流数据：Count-Min Sketch + heap。

白板实现检查点 先写出 Frequency + Selection 的不变量，再把边界条件直接编码进循环条件；完成后用一个正常样例 + 一个极端样例 dry-run。

30 秒口述 “我先给一个基线解法，然后利用 Frequency + Selection 去掉重复工作。关键不变量是：先计数，再在“元素-频率”上做堆、桶排序或 QuickSelect；频率上界是 n，桶解法可 $O(n)$ 。最终复杂度是 $O(n) / O(n)$ 。”

题 047 · LC 973 最接近原点的 K 个点 / K Closest Points to Origin

Medium 优先级 S H 高频母题 Top-K / QuickSelect

题意概括 返回欧氏距离最小的 K 个二维点。

核心突破 比较平方距离即可，避免 sqrt；K 很小时 max-heap，批量离线可 QuickSelect。

最优路径 维护 K 大小最大堆（Python 可存负距离）或 partition。

复杂度 $O(n\log k) / O(k)$

高频失误 不必要开方；堆方向写反。

追问升级 高维向量检索时为何不能直接扫描？引向 ANN/HNSW。

题 048 · LC 295 数据流的中位数 / Find Median from Data Stream

Hard 优先级 S H 高频母题 Two Heaps

题意概括 不断插入数字并随时返回中位数。

核心突破 small 最大堆存较小一半，large 最小堆存较大一半；保证大小差 ≤ 1 且所有 $\text{small} \leq \text{large}$ 。

最优路径 插入后跨堆移动；再平衡大小；奇数取大堆顶，偶数取两顶均值。

复杂度 插入 $O(\log n)$ ，查询 $O(1)$ ，空间 $O(n)$

高频失误 Python 最大堆用负数；平衡顺序和不变量。

追问升级 滑动窗口中位数需要 lazy deletion；近似分位数用 sketch。

参考实现 (Python)

```
import heapq

class MedianFinder:
    def __init__(self): self.small, self.large = [], []
    def addNum(self, x):
```

```

heapq.heappush(self.small, -x)
heapq.heappush(self.large, -heapq.heappop(self.small))
if len(self.large) > len(self.small):
    heapq.heappush(self.small, -heapq.heappop(self.large))
def findMedian(self):
    if len(self.small) > len(self.large): return -self.small[0]
    return (-self.small[0] + self.large[0]) / 2

```

30 秒口述 “我先给一个基线解法，然后利用 Two Heaps 去掉重复工作。关键不变量是：small 最大堆存较小一半，large 最小堆存较大一半；保证大小差 ≤ 1 且所有 $\text{small} \leq \text{large}$ 。最终复杂度是 插入 $O(\log n)$ ，查询 $O(1)$ ，空间 $O(n)$ 。”

题 049 · LC 23 合并 K 个升序链表 / Merge k Sorted Lists

Hard 优先级 **S** H 高频母题 K-way Merge

题意概括 合并 K 个有序链表。

核心突破 每条链表只把当前最小头节点放进最小堆，堆大小最多 K。

最优路径 初始化所有非空头；pop 最小节点接到结果；将其 next 入堆。

复杂度 $O(N \log K)$ / $O(K)$

高频失误 Python tuple 中节点不可比较，需要 tie-breaker。

追问升级 分治两两 merge 同样 $O(N \log K)$ ，常数和工程差异？

题 050 · LC 346 数据流中的移动平均值 / Moving Average from Data Stream

Easy 优先级 **A** R 公开面经样本 Fixed-size Stream State

题意概括 持续读入数值，返回最近 K 个数的平均值。

核心突破 维护固定窗口和 running sum，避免每次 $O(K)$ 重算。

最优路径 queue append； $\text{sum} += x$ ；超 K 则 $\text{sum} -= \text{popleft}$ ；返回 $\text{sum} / \text{len}(\text{queue})$ 。

复杂度 每次 $O(1)$ / $O(K)$

高频失误 前 K-1 个数据时分母是当前长度。

追问升级 高吞吐时用 ring buffer；窗口按时间而非条数如何处理？

面经证据 公开 Meta MLE 面经出现 Moving Average from Data Stream。

题 051 · LC 528 按权重随机选择 / Random Pick with Weight

Medium 优先级 **S** R 公开面经样本 Prefix Weight + Sampling

题意概括 按给定正权重比例随机返回下标。

核心突破 把权重做前缀和，相当于把 $[1, \text{total}]$ 切成不同长度区间；随机点落在哪个区间就选哪个下标。

最优路径 prefix；随机整数 $r \in [1, \text{total}]$ ；lower_bound 找第一个 $\text{prefix} \geq r$ 。

复杂度 构建 $O(n)$ ，采样 $O(\log n)$ ，空间 $O(n)$

高频失误 随机区间边界；浮点随机误差。

追问升级 权重动态更新：Fenwick Tree/Segment Tree；Alias Method 做 $O(1)$ 采样。

面经证据 公开 Meta MLE 面经出现 Random Pick with Weight。

题 052 • LC 470 用 Rand7 实现 Rand10 / Implement Rand10() Using Rand7()

Medium 优先级 S R 公开面经样本 Rejection Sampling

题意概括 只能调用均匀 Rand7，构造均匀 Rand10。

核心突破 两次 Rand7 产生 49 个等概率状态；取前 40 个映射到 10 类，剩余 9 个拒绝重采样。

最优路径 $x = (\text{rand7}() - 1) * 7 + \text{rand7}()$ ；若 $x \leq 40$ 返回 $1 + (x - 1) \% 10$ ，否则重试。

复杂度 期望 $O(1)$ 次调用 / $O(1)$

高频失误 直接取模 $49 \% 10$ 会偏；必须从等概率状态空间出发。

追问升级 如何复用拒绝区间提高随机数利用率？从 randM 到 randN 的一般方法？

面经证据 公开 LinkedIn MLE 面经出现与 rand7 -> rand10 高度同构的拒绝采样题。

参考实现 (Python)

```
def rand10():
    while True:
        x = (rand7() - 1) * 7 + rand7() # 1..49 等概率
        if x <= 40:
            return 1 + (x - 1) % 10
```

30 秒口述 “我先给一个基线解法，然后利用 Rejection Sampling 去掉重复工作。关键不变量是：两次 Rand7 产生 49 个等概率状态；取前 40 个映射到 10 类，剩余 9 个拒绝重采样。最终复杂度是 期望 $O(1)$ 次调用 / $O(1)$ 。”

题 053 • LC 912 排序数组 / Sort an Array

Medium 优先级 S R 公开面经样本 Sorting Implementation

题意概括 手写一种 $O(n \log n)$ 级别排序并讨论最坏情况。

核心突破 算法岗应同时掌握 merge sort 的稳定最坏 $O(n \log n)$ 与 randomized quicksort 的原地平均 $O(n \log n)$ 。

最优路径 归并：分治+merge；快排：随机 pivot + partition。

复杂度 Merge $O(n \log n) / O(n)$ ；Quick 平均 $O(n \log n) / O(\log n)$ 栈

高频失误 固定 pivot 遇有序数组退化；递归深度。

追问升级 稳定性、外排序、并行排序、Top-K 是否需要完整排序？

面经证据 近期国内算法岗公开面经仍出现手写快速排序/排序实现。

第 7 章 二叉树与 Tree DP

树题的统一语言是递归返回值。最关键的是把“当前答案”和“向父节点返回的信息”分开。

统一识别 先定义 dfs(node) 的返回语义，再写左右子树，再合并。

核心原型：DFS / BFS / Post-order DP / BST

```
# Tree DP
def dfs(node):
    if not node: return BASE
    left = dfs(node.left)
    right = dfs(node.right)
    update_global(node, left, right)
    return value_for_parent(node, left, right)
```

题号	LeetCode	难度	级别	核心模式
54	104	Easy	S	Tree DFS Return Value
55	102	Medium	S	BFS by Level
56	226	Easy	B	Recursive Structural Transform
57	543	Easy	S	Post-order Tree DP
58	110	Easy	A	Early-stop Tree DP
59	98	Medium	S	Range Invariant / Inorder
60	230	Medium	S	Ordered Inorder
61	236	Medium	S	Recursive Information Merge
62	199	Medium	A	BFS/DFS with Depth
63	105	Medium	S	Divide & Conquer + Index Map
64	124	Hard	S	Tree DP with Global Answer
65	314	Medium	A	BFS + Column Coordinate

题 054 · LC 104 二叉树最大深度 / Maximum Depth of Binary Tree

Easy 优先级 S H 高频母题 Tree DFS Return Value

题意概括 求二叉树最大深度。

核心突破 先定义 dfs(node) 返回“以 node 为根的子树深度”，递推自然得到 1+max(left,right)。

最优路径 空节点返回 0；递归左右；返回 1+max。

复杂度 O(n) / O(h)

高频失误 复杂度空间是递归高度 h，不总是 logn。

追问升级 BFS 层序也可；最小深度有哪些坑？

白板实现检查点 先写出 Tree DFS Return Value 的不变量，再把边界条件直接编码进循环条件；完成后用一个正常样例 + 一个极端样例 dry-run。

30 秒口述 “我先给一个基线解法，然后利用 Tree DFS Return Value 去掉重复工作。关键不变量是：先定义 dfs(node) 返回“以 node 为根的子树深度”，递推自然得到 $1 + \max(\text{left}, \text{right})$ 。最终复杂度是 $O(n) / O(h)$ 。”

题 055 • LC 102 二叉树层序遍历 / Binary Tree Level Order Traversal

Medium 优先级 S H 高频母题 BFS by Level

题意概括 按层返回树节点值。

核心突破 queue 表示当前未处理前沿；每轮固定读取当前 queue 长度，就能划分层。

最优路径 while q: level_size=len(q); 循环弹出并加入子节点。

复杂度 $O(n) / O(w)$

高频失误 边遍历边用动态 len(q) 会把下一层混入。

追问升级 锯齿层序、右视图、多源 BFS 是同一队列思想。

参考实现 (Python)

```
from collections import deque

def level_order(root):
    if not root: return []
    q = deque([root]); ans = []
    while q:
        level = []
        for _ in range(len(q)):
            node = q.popleft(); level.append(node.val)
            if node.left: q.append(node.left)
            if node.right: q.append(node.right)
        ans.append(level)
    return ans
```

30 秒口述 “我先给一个基线解法，然后利用 BFS by Level 去掉重复工作。关键不变量是：queue 表示当前未处理前沿；每轮固定读取当前 queue 长度，就能划分层。最终复杂度是 $O(n) / O(w)$ 。”

题 056 • LC 226 翻转二叉树 / Invert Binary Tree

Easy 优先级 B H 高频母题 Recursive Structural Transform

题意概括 交换每个节点的左右子树。

核心突破 树的结构性变换常可写成“先处理子树，再组合”或直接交换后递归。

最优路径 若 node: node.left, node.right = dfs(right), dfs(left)。

复杂度 $O(n) / O(h)$

高频失误 没有必要额外建树；注意是否允许原地修改。

追问升级 如何判断两棵树镜像？

题 057 • LC 543 二叉树的直径 / Diameter of Binary Tree

Easy 优先级 S H 高频母题 Post-order Tree DP

题意概括 求任意两节点间最长路径边数。

核心突破 dfs 向父节点返回“单边最大深度”；全局答案用 left_depth+right_depth，两者语义不同。

最优路径 后序求 left/right; ans=max(ans, left+right); return 1+max(left, right)。

复杂度 $O(n) / O(h)$

高频失误 把返回值直接当直径导致父节点无法使用。

追问升级 LC124 最大路径和只是把“深度”换成“最大向上贡献”。

参考实现 (Python)

```
def diameter_of_binary_tree(root):
    ans = 0
    def dfs(node):
        nonlocal ans
        if not node: return 0
        l, r = dfs(node.left), dfs(node.right)
        ans = max(ans, l+r)
        return 1 + max(l,r)
    dfs(root)
    return ans
```

30 秒口述 “我先给一个基线解法，然后利用 Post-order Tree DP 去掉重复工作。关键不变量是：dfs 向父节点返回“单边最大深度”；全局答案用 left_depth+right_depth，两者语义不同。最终复杂度是 $O(n) / O(h)$ 。”

题 058 • LC 110 平衡二叉树 / Balanced Binary Tree

Easy 优先级 A H 高频母题 Early-stop Tree DP

题意概括 判断每个节点左右子树高度差不超过 1。

核心突破 不要对每个节点重复求高度 $O(n^2)$ ；一次后序同时返回高度，失衡则返回哨兵 -1。

最优路径 left=dfs；若 -1 早停；right 同理；差>1 返回 -1，否则 1+max。

复杂度 $O(n) / O(h)$

高频失误 朴素 top-down 重复计算高度。

追问升级 如何扩展到 AVL 插入后的旋转？

题 059 • LC 98 验证二叉搜索树 / Validate Binary Search Tree

Medium 优先级 S H 高频母题 Range Invariant / Inorder

题意概括 判断树是否满足严格 BST。

核心突破 每个节点不只与父节点比较，而要满足祖先传下来的 (low,high) 全局范围。

最优路径 dfs(node,low,high)；若 val 不在范围返回 False；左更新 high，右更新 low。

复杂度 $O(n) / O(h)$

高频失误 只检查 node.left<node<node.right 会漏祖先约束；重复值规则。

追问升级 中序递增法；允许重复时规则如何定义？

参考实现 (Python)

```
def is_valid_bst(root):
    def dfs(node, low, high):
        if not node: return True
        if not (low < node.val < high): return False
        return dfs(node.left, low, node.val) and dfs(node.right, node.val, high)
    return dfs(root, float('-inf'), float('inf'))
```


30 秒口述 “我先给一个基线解法，然后利用 Range Invariant / Inorder 去掉重复工作。关键不变量是：每个节点不只与父节点比较，而满足祖先传下来的 (low,high) 全局范围。最终复杂度是 $O(n) / O(h)$ 。”

题 060 · LC 230 BST 中第 K 小的元素 / Kth Smallest in BST

Medium 优先级 **S** H 高频母题 Ordered Inorder

题意概括 BST 中序遍历天然升序，第 K 个访问节点即答案。

核心突破 可以递归计数，也可显式栈在找到第 K 个时提前停止。

最优路径 一路压左；pop 访问并 $k--$ ； $k=0$ 返回；转右。

复杂度 $O(h+k) / O(h)$

高频失误 完整遍历后排序浪费 BST 性质。

追问升级 频繁 kth 查询：节点维护 subtree size，变成 Order Statistic Tree。

题 061 · LC 236 二叉树最近公共祖先 / Lowest Common Ancestor of a Binary Tree

Medium 优先级 **S** H 高频母题 Recursive Information Merge

题意概括 给定 p,q，找最近公共祖先。

核心突破 dfs 返回“当前子树是否找到了 p/q/祖先”；若左右分别非空，当前节点就是汇合点。

最优路径 若 node 为 None/p/q 返回 node；left=dfs,right=dfs；若都非空返回 node，否则返回非空者。

复杂度 $O(n) / O(h)$

高频失误 题目是否保证 p,q 存在；BST 版本可利用值域。

追问升级 多次 LCA 查询：Euler Tour + RMQ / binary lifting。

白板实现检查点 先写出 Recursive Information Merge 的不变量，再把边界条件直接编码进循环条件；完成后用一个正常样例 + 一个极端样例 dry-run。

30 秒口述 “我先给一个基线解法，然后利用 Recursive Information Merge 去掉重复工作。关键不变量是：dfs 返回“当前子树是否找到了 p/q/祖先”；若左右分别非空，当前节点就是汇合点。最终复杂度是 $O(n) / O(h)$ 。”

题 062 · LC 199 二叉树右视图 / Binary Tree Right Side View

Medium 优先级 **A** H 高频母题 BFS/DFS with Depth

题意概括 从右侧看每层可见的最右节点。

核心突破 BFS 每层最后一个，或 DFS 先右后左并在第一次到达某深度时记录。

最优路径 层序遍历，在 $i==level_size-1$ 时 append。

复杂度 $O(n) / O(w)$

高频失误 把节点值唯一性当条件。

追问升级 左视图、锯齿、垂序遍历。

题 063 · LC 105 前序与中序构造二叉树 / Construct Binary Tree from Preorder and Inorder Traversal

Medium 优先级 **S** H 高频母题 Divide & Conquer + Index Map

题意概括 根据无重复前序和中序遍历重建树。

核心突破 前序首元素是根；中序中根的位置把左右子树切开。用 hash 避免每次线性找根。

最优路径 `index={val:i}`；递归传区间或维护 preorder 指针；根据中序边界构造左右。

复杂度 $O(n) / O(n)$

高频失误 切片会额外 $O(n^2)$ 风险；区间长度计算。

追问升级 若有重复值？信息是否足以唯一重建？

题 064 · LC 124 二叉树中的最大路径和 / Binary Tree Maximum Path Sum

Hard 优先级 **S** **H** 高频母题 Tree DP with Global Answer

题意概括 路径可从任意节点开始结束但不能重复节点，求最大和。

核心突破 向父节点只能贡献一条分支，因此 `return node.val+max(0,left,right)`；全局答案可同时取左右两条正贡献。

最优路径 后序；`left=max(0,dfs(left))`；`right` 同；`ans=max(ans,val+left+right)`；`return val+max(left,right)`。

复杂度 $O(n) / O(h)$

高频失误 全负树时 `ans` 初始化 0 会错；返回值不能包含左右两支。

追问升级 路径长度限制、必须经过根、最大乘积如何变化？

参考实现 (Python)

```
def max_path_sum(root):
    ans = float('-inf')
    def dfs(node):
        nonlocal ans
        if not node: return 0
        l = max(0, dfs(node.left)); r = max(0, dfs(node.right))
        ans = max(ans, node.val + l + r)
        return node.val + max(l, r)
    dfs(root)
    return ans
```

30 秒口述 “我先给一个基线解法，然后利用 Tree DP with Global Answer 去掉重复工作。关键不变量是：向父节点只能贡献一条分支，因此 `return node.val+max(0,left,right)`；全局答案可同时取左右两条正贡献。最终复杂度是 $O(n) / O(h)$ 。”

题 065 · LC 314 二叉树垂序遍历 / Binary Tree Vertical Order Traversal

Medium 优先级 **A** **R** 公开面经样本 BFS + Column Coordinate

题意概括 按列从左到右输出节点，同列保持从上到下/从左到右顺序。

核心突破 给 root 列坐标 0，左 -1 右 +1；BFS 天然保留层级顺序。

最优路径 `queue(node,col)`；`columns[col].append(val)`；跟踪 `min_col,max_col`。

复杂度 $O(n) / O(n)$

高频失误 若用 DFS，同层顺序可能需额外排序。

追问升级 LC987 要同时按 row/value 排序，数据结构如何升级？

面经证据 公开 Meta ML/MLE 面经出现 Vertical Order Traversal 同类题。

第 8 章 图、BFS/DFS、拓扑与并查集

搜索、依赖、连通性与最短路的统一框架。推荐/图学习/知识图谱岗位尤其重要。

统一识别 能否到达用 DFS/BFS；依赖用拓扑；动态连通用并查集；无权最短步数用 BFS。

核心原型：Graph Traversal / Topological Sort / Union-Find

```
# Kahn 拓扑排序
q = deque(v for v in V if indegree[v] == 0)
order = []
while q:
    u = q.popleft(); order.append(u)
    for v in graph[u]:
        indegree[v] -= 1
        if indegree[v] == 0: q.append(v)
```

题号	LeetCode	难度	级别	核心模式
66	200	Medium	S	Grid Connected Components
67	695	Medium	S	DFS Aggregation
68	133	Medium	A	DFS/BFS + Node Map
69	207	Medium	S	Cycle Detection / Topological Sort
70	210	Medium	S	Topological Ordering
71	127	Hard	S	Shortest Path BFS
72	994	Medium	S	Multi-source BFS
73	417	Medium	A	Reverse Reachability
74	721	Medium	S	Union-Find
75	684	Medium	A	Union-Find Cycle Detection
76	841	Medium	A	Reachability
77	399	Medium	A	Weighted Graph

题 066 · LC 200 岛屿数量 / Number of Islands

Medium 优先级 S H 高频母题 Grid Connected Components

题意概括 0/1 网格中统计四方向连通的陆地块数量。

核心突破 每遇到未访问陆地就发现一个新连通分量，并 DFS/BFS 把整块标记。

最优路径 双层扫描；grid[r][c]=='1' 时 ans++ 并 flood fill。

复杂度 O(mn) / O(mn) 最坏栈/队列

高频失误 访问标记晚于入队会重复加入；递归深度风险。

追问升级 并查集版本；动态加陆地 LC305。

参考实现 (Python)

```
def num_islands(grid):
    m,n=len(grid),len(grid[0]); ans=0
    def dfs(r,c):
```

```

    if r<0 or r>=m or c<0 or c>=n or grid[r][c]!='1': return
    grid[r][c]='0'
    dfs(r+1,c); dfs(r-1,c); dfs(r,c+1); dfs(r,c-1)
    for r in range(m):
        for c in range(n):
            if grid[r][c]=='1': ans+=1; dfs(r,c)
    return ans

```

30 秒口述 “我先给一个基线解法，然后利用 Grid Connected Components 去掉重复工作。关键不变量是：每遇到未访问陆地就发现一个新连通分量，并 DFS/BFS 把整块标记。最终复杂度是 $O(mn)$ / $O(mn)$ 最坏栈/队列。”

题 067 · LC 695 岛屿的最大面积 / Max Area of Island

Medium 优先级 S R 公开面经样本 DFS Aggregation

题意概括 求最大连通陆地区域大小。

核心突破 和岛屿数量同一母题，只是 DFS 返回值从“标记”升级为“当前连通块节点数”。

最优路径 dfs 越界/水返回 0；标记访问；return 1+四邻 dfs；取最大。

复杂度 $O(mn)$ / $O(mn)$ 最坏

高频失误 忘记先标记导致环形重复递归。

追问升级 8 邻域、带权网格、最大周长。

面经证据 公开算法岗面经中出现最大连通区域/岛屿面积同类题。

题 068 · LC 133 克隆图 / Clone Graph

Medium 优先级 A H 高频母题 DFS/BFS + Node Map

题意概括 深拷贝一个可能有环的无向图。

核心突破 图有环，必须在递归进入邻居前建立 old->new 映射，否则无限递归；本质和 Random Pointer Copy 相同。

最优路径 若 node 在 map 返回；先创建 clone 并登记；递归/遍历复制 neighbors。

复杂度 $O(V+E)$ / $O(V)$

高频失误 把 visited 与 clone map 分成两套没必要。

追问升级 有向图、带权边、序列化图。

题 069 · LC 207 课程表 / Course Schedule

Medium 优先级 S H 高频母题 Cycle Detection / Topological Sort

题意概括 存在先修依赖，判断是否能完成所有课程。

核心突破 问题等价于有向图是否无环；Kahn 算法通过不断删除入度 0 节点验证。

最优路径 建 adjacency/indegree；queue 入度 0；弹出并减邻居入度；processed==n。

复杂度 $O(V+E)$ / $O(V+E)$

高频失误 边方向搞反不一定影响“有无环”但影响输出语义；重复边。

追问升级 DFS 三色标记；返回一个可行课程顺序 LC210。

参考实现 (Python)

```

from collections import deque

def can_finish(n, prerequisites):
    g=[[] for _ in range(n)]; indeg=[0]*n
    for course, pre in prerequisites:
        g[pre].append(course); indeg[course]+=1
    q=deque(i for i,d in enumerate(indeg) if d==0); seen=0
    while q:
        u=q.popleft(); seen+=1
        for v in g[u]:
            indeg[v]-=1
            if indeg[v]==0: q.append(v)
    return seen==n

```

30 秒口述 “我先给一个基线解法，然后利用 Cycle Detection / Topological Sort 去掉重复工作。关键不变量是：问题等价于有向图是否无环；Kahn 算法通过不断删除入度 0 节点验证。最终复杂度是 $O(V+E)$ / $O(V+E)$ 。”

题 070 · LC 210 课程表 II / Course Schedule II

Medium 优先级 S H 高频母题 Topological Ordering

题意概括 在无环时返回任意合法课程顺序。

核心突破 与 LC207 同模板，只需记录 Kahn 弹出顺序；若最终数量不足 n 则有环。

最优路径 同上，`order.append(node)`。

复杂度 $O(V+E)$ / $O(V+E)$

高频失误 只判断入度但忘记返回空数组表示环。

追问升级 若要求字典序最小拓扑序？用最小堆。

题 071 · LC 127 单词接龙 / Word Ladder

Hard 优先级 S H 高频母题 Shortest Path BFS

题意概括 每次改变一个字符且中间词必须存在，求最短转换长度。

核心突破 每步代价相同，所以是无权图最短路；关键是高效生成邻居和尽早 visited。

最优路径 BFS；对每个位置尝试 26 字母或构建 wildcard pattern；首次到 end 即最短。

复杂度 朴素 $O(NL*26)$ 级 / $O(NL)$

高频失误 把词加入 visited 太晚造成指数重复；返回长度定义。

追问升级 双向 BFS 为什么更快？

白板实现检查点 先写出 Shortest Path BFS 的不变量，再把边界条件直接编码进循环条件；完成后用一个正常样例 + 一个极端样例 dry-run。

30 秒口述 “我先给一个基线解法，然后利用 Shortest Path BFS 去掉重复工作。关键不变量是：每步代价相同，所以是无权图最短路；关键是高效生成邻居和尽早 visited。最终复杂度是 朴素 $O(NL*26)$ 级 / $O(NL)$ 。”

题 072 · LC 994 腐烂的橘子 / Rotting Oranges

Medium 优先级 S H 高频母题 Multi-source BFS

题意概括 腐烂同时向四周传播，求全部腐烂所需最少分钟。

核心突破 所有初始腐烂点是同一时刻的 BFS 第 0 层，所以要一起入队，而不是逐个 BFS。

最优路径 统计 fresh；所有 rotten 入队；按层扩散并 fresh--；fresh=0 时返回分钟。

复杂度 $O(mn)$ / $O(mn)$

高频失误 分钟数 off-by-one；没有新鲜橘子应返回 0。

追问升级 火灾扩散、最近距离、多源最短路。

题 073 · LC 417 太平洋大西洋水流问题 / Pacific Atlantic Water Flow

Medium 优先级 A H 高频母题 Reverse Reachability

题意概括 找能流到两片海洋的格子，水只能从高到低/相等。

核心突破 从每个格子向海搜会重复；反过来从海岸向“更高或相等”格子搜索，两组可达集合取交集。

最优路径 分别以 Pacific 边界和 Atlantic 边界做 DFS/BFS；交集。

复杂度 $O(mn)$ / $O(mn)$

高频失误 反向条件写反；边界重复入队。

追问升级 一般图中多个终点集合的反向可达性。

题 074 · LC 721 账户合并 / Accounts Merge

Medium 优先级 S H 高频母题 Union-Find

题意概括 账户若共享邮箱则属于同一人，合并邮箱集合。

核心突破 邮箱是连通关系节点；可把同一账户内邮箱 union，最后按 root 聚合。

最优路径 email->id/name；union(first,email)；最后 root->emails 排序输出。

复杂度 近 $O(N \alpha(N))$ + 排序

高频失误 以人名作为唯一标识错误；邮箱可能跨多账户传递连接。

追问升级 DFS 图解法；大规模实体消歧如何处理？

参考实现 (Python)

```
class DSU:
    def __init__(self, n): self.p = list(range(n)); self.sz = [1]*n
    def find(self, x):
        while x != self.p[x]:
            self.p[x] = self.p[self.p[x]]; x = self.p[x]
        return x
    def union(self, a, b):
        a, b = self.find(a), self.find(b)
        if a == b: return False
        if self.sz[a] < self.sz[b]: a, b = b, a
        self.p[b] = a; self.sz[a] += self.sz[b]; return True
```

30 秒口述 “我先给一个基线解法，然后利用 Union-Find 去掉重复工作。关键不变量是：邮箱是连通关系节点；可把同一账户内邮箱 union，最后按 root 聚合。最终复杂度是 近 $O(N \alpha(N))$ + 排序。”

题 075 · LC 684 冗余连接 / Redundant Connection

Medium 优先级 A H 高频母题 Union-Find Cycle Detection

题意概括 一棵树多加一条边，找导致环的边。

核心突破 逐边加入并查集；若一条边两端已经在同一集合，则它闭合了环。

最优路径 初始化 parent；对 edge 若 $\text{find}(u) == \text{find}(v)$ 返回，否则 union。

复杂度 近 $O(E \alpha(V)) / O(V)$

高频失误 路径压缩、按秩非必要但应会；节点编号范围。

追问升级 有向图版本 LC685 更复杂在哪？

题 076 · LC 841 钥匙和房间 / Keys and Rooms

Medium 优先级 A R 公开面经样本 Reachability

题意概括 每个房间提供其他房间钥匙，判断能否访问全部房间。

核心突破 把房间视作节点、钥匙视作有向边，就是从 0 开始的可达性。

最优路径 DFS/BFS + visited，最终 $\text{len}(\text{visited}) == n$ 。

复杂度 $O(V+E) / O(V)$

高频失误 重复钥匙； $n=0/1$ 边界。

追问升级 若钥匙有消耗次数/门有条件，状态空间如何变化？

面经证据 公开 2025 面经样本出现 Keys and Rooms。

题 077 · LC 399 除法求值 / Evaluate Division

Medium 优先级 A H 高频母题 Weighted Graph

题意概括 给出 $a/b=k$ 的关系，回答若干 x/y 。

核心突破 把变量作节点， $a \rightarrow b$ 权重 k ， $b \rightarrow a$ 权重 $1/k$ ；查询就是找一条路径并累乘权重。

最优路径 建邻接表；每个 query DFS/BFS，沿边乘 product；不可达返回 -1。

复杂度 每查询 $O(V+E) / O(V)$

高频失误 浮点误差； $x==x$ 但变量未出现时仍应 -1。

追问升级 Weighted Union-Find 可把多查询加速。

第 9 章 回溯、搜索树与 Trie

本质是枚举“决策树”并通过剪枝减少状态空间。

统一识别 固定模板：选择 -> 递归 -> 撤销；Hard 题的关键通常在剪枝顺序和去重。

核心原型：Backtracking / Pruning / Trie

```
# 回溯
def dfs(state, start):
    if done(state):
        collect(state); return
    for choice in candidates(start):
        if invalid(choice): continue
        make(choice)
        dfs(next_state, next_start)
        undo(choice)
```

题号	LeetCode	难度	级别	核心模式
78	78	Medium	S	Backtracking Decision Tree
79	46	Medium	S	Backtracking with Used Set
80	39	Medium	S	Backtracking + Reuse
81	90	Medium	A	Sort + Same-level Dedup
82	79	Medium	S	Grid Backtracking
83	131	Medium	A	Partition Backtracking
84	51	Hard	A	Constraint Backtracking
85	208	Medium	S	Prefix Tree
86	698	Medium	S	Backtracking + Strong Pruning

题 078 · LC 78 子集 / Subsets

Medium 优先级 S H 高频母题 Backtracking Decision Tree

题意概括 返回集合的所有子集。

核心突破 每个元素有选/不选两种决策；也可用 start index 表示下一步可选范围，每个中间状态都是一个答案。

最优路径 dfs(start)：先 append path copy；for i from start：选 nums[i]，递归 i+1，撤销。

复杂度 $O(2^n \cdot n)$ 输出级 / $O(n)$ 路径

高频失误 append path 而不是 path.copy 导致结果引用同一对象。

追问升级 有重复元素 LC90；位掩码枚举。

题 079 · LC 46 全排列 / Permutations

Medium 优先级 S H 高频母题 Backtracking with Used Set

题意概括 返回无重复数组的所有排列。

核心突破 每层选择一个尚未使用的元素；深度就是当前位置。

最优路径 `used[]`；for 每个未使用 `i`：标记、加入、递归、撤销。

复杂度 $O(n \cdot n!) / O(n)$

高频失误 忘记撤销 `used`；用 `membership path` 导致额外 $O(n)$ 。

追问升级 有重复元素如何排序+层内去重？

题 080 · LC 39 组合总和 / Combination Sum

Medium 优先级 **S** H 高频母题 Backtracking + Reuse

题意概括 候选正数可重复使用，找和为 `target` 的组合。

核心突破 通过 `start` 控制组合而非排列；选当前 `i` 后递归仍从 `i` 开始表示可重复。

最优路径 排序可剪枝；`dfs(start, remain)`；`remain=0` 收集，`nums[i]>remain` break。

复杂度 指数级 / $O(\text{target}/\text{minVal})$ 深度

高频失误 递归 `i+1` 会错误禁止重复；不设 `start` 会产生排列重复。

追问升级 候选含负数时为何会出现无限搜索问题？

题 081 · LC 90 子集 II / Subsets II

Medium 优先级 **A** H 高频母题 Sort + Same-level Dedup

题意概括 输入含重复元素，返回不重复子集。

核心突破 排序后，重复元素只有在“同一递归层且不是第一个候选”时跳过；不同深度仍可选择。

最优路径 sort；for `i` in range(`start`, `n`): if `i>start` and `nums[i]==nums[i-1]`:continue。

复杂度 $O(2^n) / O(n)$

高频失误 把所有重复值都跳过会漏 `[2,2]`。

追问升级 `Permutations II` 的层内去重有什么不同？

题 082 · LC 79 单词搜索 / Word Search

Medium 优先级 **S** H 高频母题 Grid Backtracking

题意概括 在字符网格中通过上下左右且不重复使用单元格拼出单词。

核心突破 状态包含位置与 `word index`；选择邻居前临时标记当前格，返回后恢复。

最优路径 从每格作为起点 `dfs(r,c,i)`；字符不符返回；`i` 到末尾成功；四方向搜索。

复杂度 $O(mn \cdot 4^L)$ 上界 / $O(L)$

高频失误 成功时恢复标记的控制流；边界与重复使用。

追问升级 用 Trie 同时搜索很多单词 LC212。

题 083 · LC 131 分割回文串 / Palindrome Partitioning

Medium 优先级 A H 高频母题 Partition Backtracking

题意概括 把字符串切成若干回文子串，返回所有切法。

核心突破 每层枚举下一刀 end；只有 s[start:end+1] 是回文才进入下一层。

最优路径 dfs(start); for end; if palindrome: path.append; dfs(end+1); pop。

复杂度 指数级，回文预处理可 $O(n^2)$

高频失误 每次切片+回文判断造成较大常数；忘记复制 path。

追问升级 用 DP 预计算 palindrome；最少切割 LC132。

题 084 · LC 51 N 皇后 / N-Queens

Hard 优先级 A H 高频母题 Constraint Backtracking

题意概括 在 $N \times N$ 棋盘放 N 个皇后互不攻击。

核心突破 逐行放置；列、主对角 $r-c$ 、副对角 $r+c$ 三个集合即可 $O(1)$ 判断冲突。

最优路径 dfs(row)；枚举 col，若三个约束均空则放置、递归、撤销。

复杂度 约 $O(N!)$ / $O(N)$

高频失误 对角线编码；构造棋盘输出耗时。

追问升级 只求数量 LC52；bitmask 如何显著加速？

题 085 · LC 208 实现 Trie / Implement Trie

Medium 优先级 S H 高频母题 Prefix Tree

题意概括 支持插入单词、完整查找与前缀查找。

核心突破 把公共前缀共享成路径；节点需要 children 映射与 is_end 标记区分单词和前缀。

最优路径 insert 逐字符创建；search 走完检查 is_end；startsWith 只需路径存在。

复杂度 每操作 $O(L)$ / 总字符节点数

高频失误 只靠 children 不能区分 app 与 apple 中的 app 是否已插入。

追问升级 Autocomplete、前缀 Top-K、压缩 Trie/Radix Tree。

题 086 · LC 698 划分为 K 个相等子集 / Partition to K Equal Sum Subsets

Medium 优先级 S R 公开面经样本 Backtracking + Strong Pruning

题意概括 把数组恰好分成 K 组且每组和相等。

核心突破 先检查总和可整除；目标 bucket=sum/k。优先放大数，按桶搜索并用“空桶等价”剪枝。

最优路径 nums 降序；buckets[k]；dfs(i) 尝试放入不超 target 的桶；若放入空桶失败可 break。

复杂度 最坏指数级 / $O(k+n)$

高频失误 不排序导致爆炸；相同桶状态重复搜索。

追问升级 状态压缩 DP；为什么“空桶 break”正确？

面经证据 公开 LinkedIn MLE full loop 出现 Partition to K Equal Sum Subsets 同类题。

参考实现 (Python)

```
def can_partition_k_subsets(nums, k):
    total=sum(nums)
    if total%k: return False
    target=total//k; nums.sort(reverse=True)
    if nums and nums[0]>target: return False
    buckets=[0]*k
    def dfs(i):
        if i==len(nums): return True
        x=nums[i]
        for j in range(k):
            if buckets[j]+x<=target:
                buckets[j]+=x
                if dfs(i+1): return True
                buckets[j]-=x
            if buckets[j]==0: break
        return False
    return dfs(0)
```

30 秒口述 “我先给一个基线解法，然后利用 Backtracking + Strong Pruning 去掉重复工作。关键不变量是：先检查总和可整除；目标 bucket=sum/k。优先放大数，按桶搜索并用“空桶等价”剪枝。最终复杂度是 最坏指数级 / $O(k+n)$ 。”

第 10 章 动态规划、贪心与区间

算法岗的区分度模块。DP 的核心是压缩历史；贪心的核心是证明局部选择不会破坏全局最优。

统一识别 先写状态含义和转移，再谈初始化与空间优化；不要一上来猜 dp 数组。

核心原型： DP / Knapsack / Sequence DP / Greedy / Intervals

```
# DP 四问
# 1. state 表示什么？
# 2. transition 从哪些更小状态来？
# 3. base case 是什么？
# 4. 计算顺序能否保证前驱已就绪？
for state in order:
    dp[state] = best(previous_states)
```

题号	LeetCode	难度	级别	核心模式
87	70	Easy	S	1D DP
88	198	Medium	S	Take/Skip DP
89	322	Medium	S	Unbounded Knapsack
90	300	Medium	S	DP + Patience Binary Search
91	1143	Medium	S	2D Sequence DP
92	72	Medium	S	2D Alignment DP
93	139	Medium	S	Prefix DP
94	416	Medium	S	0/1 Knapsack
95	152	Medium	A	Dual-state DP
96	121	Easy	S	One-pass State / Greedy
97	56	Medium	S	Sort + Sweep
98	57	Medium	A	Three-phase Interval Scan
99	435	Medium	A	Interval Scheduling Greedy
100	45	Medium	S	Layered Greedy

题 087 · LC 70 爬楼梯 / Climbing Stairs

Easy 优先级 S H 高频母题 1D DP

题意概括 每次爬 1 或 2 阶，求到 n 阶方案数。

核心突破 到 i 的最后一步来自 i-1 或 i-2，因此 $f(i)=f(i-1)+f(i-2)$ 。

最优路径 滚动两个变量即可。

复杂度 $O(n)$ / $O(1)$

高频失误 $n=1/2$ 初始化；方案数增长。

追问升级 一次可跳 1..k 阶？矩阵快速幂如何降到 $O(\log n)$ ？

题 088 • LC 198 打家劫舍 / House Robber

Medium 优先级 S H 高频母题 Take/Skip DP

题意概括 不能偷相邻房屋，求最大金额。

核心突破 决定第 i 间时只有两种：不偷继承 $dp[i-1]$ ；偷则 $dp[i-2]+nums[i]$ 。

最优路径 $prev2, prev1$ 滚动更新 $cur = \max(prev1, prev2+x)$ 。

复杂度 $O(n) / O(1)$

高频失误 把贪心“挑局部大值”当正确。

追问升级 环形房屋 LC213；树形房屋 LC337。

题 089 • LC 322 零钱兑换 / Coin Change

Medium 优先级 S H 高频母题 Unbounded Knapsack

题意概括 给硬币面额，可无限使用，求凑 $amount$ 的最少硬币数。

核心突破 $dp[a]$ 表示凑 a 的最少硬币；从所有 $coin$ 的 $dp[a-coin]+1$ 转移。

最优路径 $dp = [inf] * (amount+1)$; $dp[0]=0$; for a 1..amount for $c \leq a$ 更新。

复杂度 $O(amount * C) / O(amount)$

高频失误 inf 与不可达；组合数问题的循环顺序不同。

追问升级 Coin Change II 为什么循环顺序影响排列/组合？

参考实现 (Python)

```
def coin_change(coins, amount):
    dp = [float('inf')] * (amount+1); dp[0]=0
    for a in range(1, amount+1):
        for c in coins:
            if c <= a:
                dp[a] = min(dp[a], dp[a-c]+1)
    return -1 if dp[amount]==float('inf') else dp[amount]
```

30 秒口述 “我先给一个基线解法，然后利用 Unbounded Knapsack 去掉重复工作。关键不变量是： $dp[a]$ 表示凑 a 的最少硬币；从所有 $coin$ 的 $dp[a-coin]+1$ 转移。最终复杂度是 $O(amount * C) / O(amount)$ 。”

题 090 • LC 300 最长递增子序列 / Longest Increasing Subsequence

Medium 优先级 S H 高频母题 DP + Patience Binary Search

题意概括 求最长严格递增子序列长度。

核心突破 $O(n^2)$ DP 易懂；最优用 $tails[len]$ 记录该长度递增序列可达到的最小尾值，越小越利于延伸。

最优路径 对 x 用 $lower_bound$ 找第一个 $\geq x$ 的 $tails$ 位置替换；若不存在则 $append$ 。

复杂度 $O(n \log n) / O(n)$

高频失误 $tails$ 不是实际 LIS；严格递增用 $lower_bound$ ，非降用 $upper_bound$ 。

追问升级 如何恢复实际序列？

参考实现 (Python)

```
from bisect import bisect_left

def length_of_lis(nums):
```

```

tails=[]
for x in nums:
    i=bisect_left(tails,x)
    if i==len(tails): tails.append(x)
    else: tails[i]=x
return len(tails)

```

30 秒口述 “我先给一个基线解法，然后利用 DP + Patience Binary Search 去掉重复工作。关键不变量是： $O(n^2)$ DP 易懂；最优用 `tails[len]` 记录该长度递增序列可达到的最小尾值，越小越利于延伸。最终复杂度是 $O(n \log n) / O(n)$ 。”

题 091 · LC 1143 最长公共子序列 / Longest Common Subsequence

Medium 优先级 S H 高频母题 2D Sequence DP

题意概括 求两个字符串最长公共子序列长度。

核心突破 `dp[i][j]` 表示前 i, j 个字符的答案；末字符相同则 +1，否则丢弃一边取最大。

最优路径 if `a[i-1]==b[j-1]`: `dp=prev_diag+1` else `max(up,left)`。

复杂度 $O(mn) / O(mn)$ ，可压 $O(\min(m,n))$

高频失误 子序列不是子串；滚动数组时更新顺序。

追问升级 编辑距离、diff、序列对齐均来自相似二维 DP。

参考实现 (Python)

```

def lcs(a,b):
    prev=[0]*(len(b)+1)
    for ca in a:
        cur=[0]*(len(b)+1)
        for j,cb in enumerate(b,1):
            cur[j]=prev[j-1]+1 if ca==cb else max(prev[j],cur[j-1])
        prev=cur
    return prev[-1]

```

30 秒口述 “我先给一个基线解法，然后利用 2D Sequence DP 去掉重复工作。关键不变量是：`dp[i][j]` 表示前 i, j 个字符的答案；末字符相同则 +1，否则丢弃一边取最大。最终复杂度是 $O(mn) / O(mn)$ ，可压 $O(\min(m,n))$ 。”

题 092 · LC 72 编辑距离 / Edit Distance

Medium 优先级 S H 高频母题 2D Alignment DP

题意概括 插入、删除、替换一个字符代价 1，把 `word1` 变 `word2` 的最少操作数。

核心突破 `dp[i][j]` 表示前缀转换成本；末字符不同时比较删除、插入、替换三个前驱状态。

最优路径 字符同取 `dp[i-1][j-1]`；不同 $1+\min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])$ 。

复杂度 $O(mn) / O(mn)$ ，可压一维

高频失误 初始化 `dp[i][0]=i, dp[0][j]=j`；混淆操作方向。

追问升级 不同操作不同权重；如何回溯具体编辑脚本？

题 093 · LC 139 单词拆分 / Word Break

Medium 优先级 S H 高频母题 Prefix DP

题意概括 判断字符串能否拆成字典中的若干单词。

核心突破 $dp[i]$ 表示前 i 个字符是否可拆；只要存在 $j < i$ 且 $dp[j]$ 且 $s[j:i]$ 在字典即可。

最优路径 $dp[0]=True$ ；for i ；for j 或只枚举词长。

复杂度 $O(n^2)$ 典型 / $O(n)$

高频失误 忘记空前缀 $dp[0]=True$ ；切片成本。

追问升级 返回所有拆分方式会变回溯+记忆化 LC140。

题 094 · LC 416 分割等和子集 / Partition Equal Subset Sum

Medium 优先级 **S** H 高频母题 0/1 Knapsack

题意概括 判断数组能否分成和相等的两组。

核心突破 若总和为偶数，问题变成是否存在子集和 $target = sum/2$ ，即 0/1 背包可达性。

最优路径 $dp[0]=True$ ；对每个 num 从 $target$ 向下更新 $dp[s]=dp[s]$ or $dp[s-num]$ 。

复杂度 $O(n*target)$ / $O(target)$

高频失误 0/1 背包必须倒序更新，否则同一元素被重复使用。

追问升级 若求方案数？若元素可重复则改正序。

参考实现 (Python)

```
def can_partition(nums):
    s = sum(nums)
    if s % 2: return False
    target = s // 2; dp = [False] * (target + 1); dp[0] = True
    for x in nums:
        for t in range(target, x - 1, -1):
            dp[t] = dp[t] or dp[t - x]
    return dp[target]
```

30 秒口述 “我先给一个基线解法，然后利用 0/1 Knapsack 去掉重复工作。关键不变量是：若总和为偶数，问题变成是否存在子集和 $target = sum/2$ ，即 0/1 背包可达性。最终复杂度是 $O(n*target)$ / $O(target)$ 。”

题 095 · LC 152 乘积最大子数组 / Maximum Product Subarray

Medium 优先级 **A** H 高频母题 Dual-state DP

题意概括 连续子数组最大乘积。

核心突破 负数会让最大与最小互换，所以必须同时维护到当前位置的 max_prod 与 min_prod 。

最优路径 若 $x < 0$ 先交换 max/min ； $max = \max(x, max * x)$ ； $min = \min(x, min * x)$ ；更新答案。

复杂度 $O(n)$ / $O(1)$

高频失误 只维护最大值会错过负负得正；0 会自然重置。

追问升级 最大绝对乘积、返回区间。

题 096 · LC 121 买卖股票的最佳时机 / Best Time to Buy and Sell Stock

Easy 优先级 **S** H 高频母题 One-pass State / Greedy

题意概括 只能买一次卖一次且先买后卖，求最大利润。

核心突破 扫描到价格 p 时，只需知道此前最低买入价；当前最佳卖出利润是 $p - min_price$ 。

最优路径 `min_price=inf;ans=0`；逐个更新 `ans` 与 `min_price`。

复杂度 $O(n) / O(1)$

高频失误 先更新 `min_price` 与 `ans` 都可但要确保语义；允许负利润时通常返回 0。

追问升级 多次交易、手续费、冷冻期最终形成状态机 DP。

题 097 · LC 56 合并区间 / Merge Intervals

Medium 优先级 S R 公开面经样本 Sort + Sweep

题意概括 合并所有重叠区间。

核心突破 按起点排序后，新区间只可能与结果最后一个区间重叠。

最优路径 sort by start；若 `start <= last_end` 则扩 `last_end`，否则 append 新区间。

复杂度 $O(n \log n) / O(n)$ 输出

高频失误 端点相接算不算重叠取决于题意；原地修改。

追问升级 会议室、区间交集、扫描线。

面经证据 公开 Microsoft/Meta 等面经长期出现 Merge/Insert Interval 同类题。

参考实现 (Python)

```
def merge(intervals):
    intervals.sort(key=lambda x:x[0]); ans=[]
    for s,e in intervals:
        if not ans or s>ans[-1][1]: ans.append([s,e])
        else: ans[-1][1]=max(ans[-1][1],e)
    return ans
```

30 秒口述 “我先给一个基线解法，然后利用 Sort + Sweep 去掉重复工作。关键不变量是：按起点排序后，新区间只可能与结果最后一个区间重叠。最终复杂度是 $O(n \log n) / O(n)$ 输出。”

题 098 · LC 57 插入区间 / Insert Interval

Medium 优先级 A R 公开面经样本 Three-phase Interval Scan

题意概括 已有不重叠有序区间中插入一个新区间并保持合并。

核心突破 分三段：完全在新区间左侧；与新区间重叠并不断扩张；完全在右侧。

最优路径 先 append `end < new.start`；while `start <= new.end` 合并；append `new`；再接剩余。

复杂度 $O(n) / O(n)$ 输出

高频失误 边界是否闭区间；在循环中错误追加多次 `new`。

追问升级 如果原区间无序/有重叠，先转 LC56。

面经证据 公开 Meta ML 面经出现 Insert Interval。

题 099 · LC 435 无重叠区间 / Non-overlapping Intervals

Medium 优先级 A H 高频母题 Interval Scheduling Greedy

题意概括 删除最少区间使剩余互不重叠。

核心突破 等价于保留最多互不重叠区间；经典贪心按结束时间升序，总选最早结束者给未来留最大空间。

最优路径 sort by end; 若 $\text{start} \geq \text{current_end}$ 则保留并更新 end，否则删除计数++。

复杂度 $O(n \log n)$ / $O(1)$ 除排序

高频失误 按起点排序再随意删除不易证明；闭开区间条件。

追问升级 会议安排最大数量；带权区间需 DP。

题 100 · LC 45 跳跃游戏 II / Jump Game II

Medium 优先级 S H 高频母题 Layered Greedy

题意概括 每个位置给最大跳跃距离，求到末尾最少跳跃次数（保证可达）。

核心突破 把当前一次跳跃能覆盖的区间看成 BFS 一层；扫描层内所有点求 next_farthest，到达层边界时 jumps++。

最优路径 $\text{end}=0, \text{farthest}=0$; for $i < n-1$: $\text{farthest} = \max(\text{farthest}, i + \text{nums}[i])$; if $i == \text{end}$: $\text{jumps}++$; $\text{end} = \text{farthest}$ 。

复杂度 $O(n)$ / $O(1)$

高频失误 遍历到 $n-1$ 会多跳一次；没有理解“层边界”。

追问升级 LC55 只判断可达；若每次跳有代价则变最短路/DP。

参考实现 (Python)

```
def jump(nums):
    jumps=end=farthest=0
    for i in range(len(nums)-1):
        farthest=max(farthest, i+nums[i])
        if i==end:
            jumps+=1; end=farthest
    return jumps
```

30 秒口述 “我先给一个基线解法，然后利用 Layered Greedy 去掉重复工作。关键不变量是：把当前一次跳跃能覆盖的区间看成 BFS 一层；扫描层内所有点求 next_farthest，到达层边界时 jumps++。最终复杂度是 $O(n)$ / $O(1)$ 。”

附录 A：S 级核心题清单

若面试只有 2-3 周准备时间，优先把下列 S 级题做到“无提示写对 + 能讲追问”。S 级不是难度，而是模式覆盖率与迁移价值。

题号	LC	题目	模式
1	1	两数之和	Hash 查补数
2	49	字母异位词分组	Canonical Key
3	128	最长连续序列	Hash Set + 起点判定
4	238	除自身以外数组的乘积	Prefix/Suffix Product
5	560	和为 K 的子数组	Prefix Sum + Hash Count
7	41	缺失的第一个正数	In-place Index Mapping
12	15	三数之和	Sort + Two Pointers
13	11	盛最多水的容器	Greedy Two Pointers
14	42	接雨水	Two Pointers / Monotonic Stack
15	3	无重复字符的最长子串	Variable Window
17	76	最小覆盖子串	Need/Have Sliding Window
20	239	滑动窗口最大值	Monotonic Deque
21	206	反转链表	Pointer Rewiring
22	92	反转链表 II	Segment Reversal
23	21	合并两个有序链表	Two-list Merge
24	141	环形链表	Floyd Cycle Detection
25	142	环形链表 II	Floyd Entrance Math
26	25	K 个一组翻转链表	Chunked Reversal
30	146	LRU 缓存	Hash + Doubly Linked List
31	20	有效的括号	Stack Matching
32	155	最小栈	Augmented State
33	394	字符串解码	Stack Parser
34	739	每日温度	Monotonic Stack
35	84	柱状图中最大的矩形	Monotonic Increasing Stack
37	704	二分查找	Exact Binary Search
38	33	搜索旋转排序数组	Partitioned Monotonicity
40	34	查找元素首尾位置	Lower/Upper Bound
41	4	寻找两个正序数组的中位数	Binary Partition
42	875	爱吃香蕉的珂珂	Binary Search on Answer
43	1011	在 D 天内送达包裹的能力	Feasibility Search
45	215	数组中的第 K 个最大元素	Heap / QuickSelect
46	347	前 K 个高频元素	Frequency + Selection
47	973	最接近原点的 K 个点	Top-K / QuickSelect
48	295	数据流的中位数	Two Heaps
49	23	合并 K 个升序链表	K-way Merge
51	528	按权重随机选择	Prefix Weight + Sampling
52	470	用 Rand7 实现 Rand10	Rejection Sampling

53	912	排序数组	Sorting Implementation
54	104	二叉树最大深度	Tree DFS Return Value
55	102	二叉树层序遍历	BFS by Level
57	543	二叉树的直径	Post-order Tree DP
59	98	验证二叉搜索树	Range Invariant / Inorder
60	230	BST 中第 K 小的元素	Ordered Inorder
61	236	二叉树最近公共祖先	Recursive Information Merge
63	105	前序与中序构造二叉树	Divide & Conquer + Index Map
64	124	二叉树中的最大路径和	Tree DP with Global Answer
66	200	岛屿数量	Grid Connected Components
67	695	岛屿的最大面积	DFS Aggregation
69	207	课程表	Cycle Detection / Topological Sort
70	210	课程表 II	Topological Ordering
71	127	单词接龙	Shortest Path BFS
72	994	腐烂的橘子	Multi-source BFS
74	721	账户合并	Union-Find
78	78	子集	Backtracking Decision Tree
79	46	全排列	Backtracking with Used Set
80	39	组合总和	Backtracking + Reuse
82	79	单词搜索	Grid Backtracking
85	208	实现 Trie	Prefix Tree
86	698	划分为 K 个相等子集	Backtracking + Strong Pruning
87	70	爬楼梯	1D DP
88	198	打家劫舍	Take/Skip DP
89	322	零钱兑换	Unbounded Knapsack
90	300	最长递增子序列	DP + Patience Binary Search
91	1143	最长公共子序列	2D Sequence DP
92	72	编辑距离	2D Alignment DP
93	139	单词拆分	Prefix DP
94	416	分割等和子集	0/1 Knapsack
96	121	买卖股票的最佳时机	One-pass State / Greedy
97	56	合并区间	Sort + Sweep
100	45	跳跃游戏 II	Layered Greedy

附录 B：算法岗 “LeetCode -> 系统问题” 转换表

LeetCode 母题	工程约束变化	算法岗升级方向
LC215/347 Top-K	数据量 > 内存	K 大小堆、外部排序、分片 Local Top-K -> Global Merge
LC295 Median Stream	无限流 / 滑动窗口	双堆 + lazy deletion; 近似分位数用

LC146 LRU	线上缓存	TTL、并发锁/分段锁、LFU、分布式一致性、KV Cache 驱逐
LC208 Trie	搜索提示	prefix top-k、Radix Tree、FST、缓存热门前缀
LC973 K Closest	高维向量召回	精确扫描 -> ANN；HNSW/IVF 等索引的 recall-latency-memory trade-off
LC528/470 Sampling	训练数据/负采样	alias table、reservoir sampling、importance sampling、动态权重结构
LC207 Topological	特征/任务依赖	DAG 调度、数据 pipeline、循环依赖检测、并行执行
LC721 Union-Find	实体归并	大规模去重、entity resolution、离线批处理与增量合并

附录 C：21 天冲刺计划

时间	主题	题目	训练动作
Day 1-3	数组/Hash/前缀	1-10，重点 1/5/7	每题先讲状态压缩，再写代码
Day 4-5	双指针/滑窗	11-20，重点 12/14/15/17/20	强制写“窗口合法条件”
Day 6-7	链表/缓存	21-30，重点 21/22/26/30	纸上画指针变化
Day 8	栈/单调结构	31-36，重点 34/35	解释为何每个元素只入栈出栈一次
Day 9-10	二分	37-44，重点 38/40/41/42	统一写边界模板 + check(x)
Day 11	Heap/Streaming/Sampling	45-53	把每题追加“十亿数据怎么办”
Day 12-14	Tree	54-65	每题先写 dfs 返回语义
Day 15-16	Graph	66-77	BFS/拓扑/并查集分类训练
Day 17	Backtracking/Trie	78-86	练剪枝和去重，不追求背代码
Day 18-20	DP/Greedy	87-100	先写 state/transition/base/order
Day 21	模拟面试	随机 4 Medium + 1 Hard	限时、口述、dry-run、追问

附录 D：面试自检表

- 能否在 60 秒内说出暴力解与瓶颈？
- 是否明确说出了输入假设，而不是偷偷依赖“无重复/正数/有序”？
- 是否能用一句话定义核心不变量？
- 是否在写代码前给出时间/空间目标？
- 循环边界是否能解释，而不是靠“背模板”？
- 是否 dry-run：空输入、1 个元素、重复、极值、K=1/K=n？
- 是否主动解释复杂度的来源？
- 若数据量扩大 1000 倍，当前方案首先在哪个资源上崩？
- 若输入变成数据流，哪些状态必须增量维护？
- 能否把题目关联到推荐/检索/训练/服务中的真实场景？

参考资料与证据来源（公开资料）

LeetCode Top Interview 150 — 官方学习计划：150 道经典面试题，覆盖主要面试主题。

<https://leetcode.com/studyplan/top-interview-150/>

LinkedIn Software Engineer, Machine Learning Interview Experience — 公开个人面经：AI Coding、DSA、拒绝采样、K 等和子集、推荐系统设计。

<https://leetcode.com/discuss/post/7405614/>

TikTok MLE Phone Screen — 公开个人面经：Reverse Linked List 与 Reverse Linked List II。

<https://leetcode.com/discuss/post/2073085/>

Meta E4 MLE Interview Experience — 公开个人面经：Moving Average from Data Stream、Random Pick with Weight。

<https://leetcode.com/discuss/post/4241029/>

Google L4 Bangalore 2025 — 公开个人面经：Top-K min heap、Topological Sorting + BFS。

<https://leetcode.com/discuss/post/7240829/>

Meta SWE-ML E5 London — 公开个人面经：多轮 Medium coding + ML system design。

<https://leetcode.com/discuss/post/7066005/>

Meta ML PhD Intern Interview — 公开个人面经：Vertical Order Traversal、Insert Interval 等。

<https://leetcode.com/discuss/post/1643971/>

Google New Grad 2025 Technical Interview — 公开个人面经：拓扑排序题。

<https://leetcode.com/discuss/post/5973486/>

证据边界 公开面经是样本，不是官方题库。公司会更换题目、使用变体或内部题。本书的目标是最大化“模式覆盖率”，而不是承诺命中具体原题。

结语：从题库走向算法能力

真正稳定的面试能力来自三个层次：第一层能识别模式；第二层能证明不变量与复杂度；第三层能在约束改变后重构方案。100 道题只是训练载体。若一道 LC215 能继续讨论 streaming、memory bound、distributed Top-K；一道 LC146 能继续讨论 TTL、并发和 KV Cache；一道 LC528 能继续讨论训练负采样，那么 LeetCode 才真正转化成算法工程能力。

最终目标 40 道核心题 -> 100 道母题 -> 25 个算法原型 -> 数据规模/系统约束变体。面试时不依赖“见过原题”，而依赖可以迁移的推理结构。